

SIP Codec API Specification

Version 1.0; July 5, 2004

SIP

NOKIA

Copyright © 2004 Nokia Corporation. All rights reserved.

Nokia and Nokia Connecting People are registered trademarks of Nokia Corporation. Java and all Java-based marks are trademarks or registered trademarks of Sun Microsystems, Inc. Other product and company names mentioned herein may be trademarks or trade names of their respective owners.

Disclaimer

The information in this document is provided “as is,” with no warranties whatsoever, including any warranty of merchantability, fitness for any particular purpose, or any warranty otherwise arising out of any proposal, specification, or sample. Furthermore, information provided in this document is preliminary, and may be changed substantially prior to final release. This document is provided for informational purposes only.

Nokia Corporation disclaims all liability, including liability for infringement of any proprietary rights, relating to implementation of information presented in this document. Nokia Corporation does not warrant or represent that such use will not infringe such rights.

Nokia Corporation retains the right to make changes to this specification at any time, without notice.

License

A license is hereby granted to download and print a copy of this specification for personal use only. No other license to any other intellectual property rights is granted herein.

Contents

1	Introduction.....	7
1.1	List of Required Interfaces	7
1.2	Product Platform and/or Symbian OS Version Constraints.....	7
2	Technical Specification.....	8
2.1	Type of Interface.....	8
2.2	Interface Class Structure	8
2.2.1	Static class diagrams.....	8
2.3	Usage.....	9
2.3.1	Protocol	9
2.3.2	Error handling	10
2.3.3	Memory overhead.....	10
2.3.4	Extensions to the API.....	10
2.4	Example	10
2.5	Detailed Description	11
2.5.1	CSIPAcceptEncodingHeader	11
2.5.2	CSIPAcceptHeader	16
2.5.3	CSIPAcceptLanguageHeader	23
2.5.4	CSIPAddress.....	28
2.5.5	CSIPAllowEventsHeader.....	33
2.5.6	CSIPAllowHeader	39
2.5.7	CSIPContactHeader.....	42
2.5.8	CSIPContentDispositionHeader	49
2.5.9	CSIPContentEncodingHeader.....	54
2.5.10	CSIPContentTypeHeader.....	58
2.5.11	CSIPSeqHeader	63
2.5.12	CSIPEventHeader	68
2.5.13	CSIPExpiresHeader	74
2.5.14	CSIPExtensionHeader	76
2.5.15	CSIPFromHeader	81
2.5.16	CSIPFromToHeaderBase	86
2.5.17	CSIPHeaderBase	88
2.5.18	CSIPHostPort	93
2.5.19	CSIPParameterHeaderBase	98
2.5.20	CSIPProxyRequireHeader	101
2.5.21	CSIPRequireHeader	104
2.5.22	CSIPRouteHeader.....	107

2.5.23	CSIPRouteHeaderBase.....	111
2.5.24	CSIPSecurityClientHeader.....	112
2.5.25	CSIPSecurityHeaderBase	116
2.5.26	CSIPSecurityServerHeader	120
2.5.27	CSIPSubscriptionStateHeader.....	124
2.5.28	CSIPSupportedHeader.....	130
2.5.29	CSIPToHeader	134
2.5.30	CSIPTokenHeaderBase	138
2.5.31	CSIPUnsignedIntHeaderBase	139
2.5.32	CSIPUnsupportedHeader.....	141
2.5.33	CSIPURI.....	144
2.5.34	CURIBase.....	161
2.6	Code Architecture	163
3	Index	165
3.1	List of Constants and Enums.....	165
3.2	Alphabetical List of Classes and Functions.....	167
3.2.1	CSIPAcceptEncodingHeader	167
3.2.2	CSIPAcceptHeader	167
3.2.3	CSIPAcceptLanguageHeader	168
3.2.4	CSIPAddress.....	168
3.2.5	CSIPAllowEventsHeader.....	168
3.2.6	CSIPAllowHeader	169
3.2.7	CSIPContactHeader.....	169
3.2.8	CSIPContentDispositionHeader.....	169
3.2.9	CSIPContentEncodingHeader.....	170
3.2.10	CSIPContentTypeHeader.....	170
3.2.11	CSIPSeqHeader	171
3.2.12	CSIPEventHeader	171
3.2.13	CSIPExpiresHeader	171
3.2.14	CSIPExtensionHeader	172
3.2.15	CSIPFromHeader	172
3.2.16	CSIPFromToHeaderBase	172
3.2.17	CSIPHeaderBase	172
3.2.18	CSIPHostPort	173
3.2.19	CSIPParameterHeaderBase	173
3.2.20	CSIPProxyRequireHeader	173
3.2.21	CSIPRequireHeader	174
3.2.22	CSIPRouteHeader.....	174

3.2.23	CSIPRouteHeaderBase.....	174
3.2.24	CSIPSecurityClientHeader.....	174
3.2.25	CSIPSecurityHeaderBase	175
3.2.26	CSIPSecurityServerHeader	175
3.2.27	CSIPSubscriptionStateHeader	175
3.2.28	CSIPSupportedHeader	176
3.2.29	CSIPToHeader	176
3.2.30	CSIPTokenHeaderBase	176
3.2.31	CSIPUnsignedIntHeaderBase	176
3.2.32	CSIPUnsupportedHeader.....	177
3.2.33	CSIPURI.....	177
3.2.34	CURIBase.....	178
4	Terms and Abbreviations	179
5	References	180

Change History

July 5, 2004	Version 1.0	Initial document release

1 Introduction

The Session Initiation Protocol (SIP) Codec API provides services for encoding/decoding SIP headers and their fields. For more information on SIP, refer to *RFC 3261* [1].

The API also provides means for converting SIP messages from the UTF-8 ASCII format to the API implementation's internal structure and vice versa.

The API implementation verifies the SIP message syntax according to the SIP Backus-Naur Form (BNF) definition.

The API implementation supports serialization/deserialization of objects belonging to the SIP message structure.

The implementation of the API is provided in a shared DLL that is executed in the user's context. The API implementation creates class instances in the user's heap area, so these instances cannot be shared between different users.

Readers of this document should have a good understanding of the SIP protocol.

1.1 List of Required Interfaces

None.

1.2 Product Platform and/or Symbian OS Version Constraints

The SIP Codec API is valid for all the platforms running Symbian OS v7.0s.

2 Technical Specification

2.1 Type of Interface

The logical type of the SIP Codec API is Library API. The interface contains synchronous methods that do not block but return immediately. The interface uses only local objects.

2.2 Interface Class Structure

2.2.1 Static class diagrams

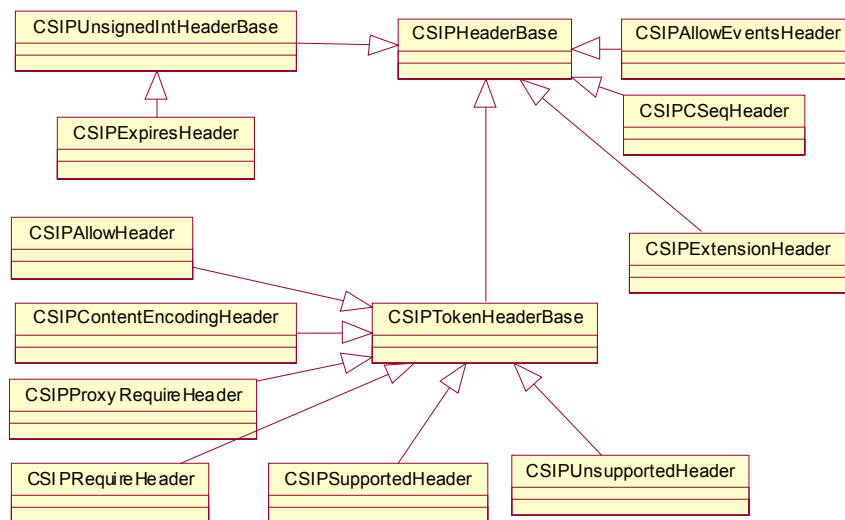


Figure 1: Relationship of SIP Codec classes (part one)

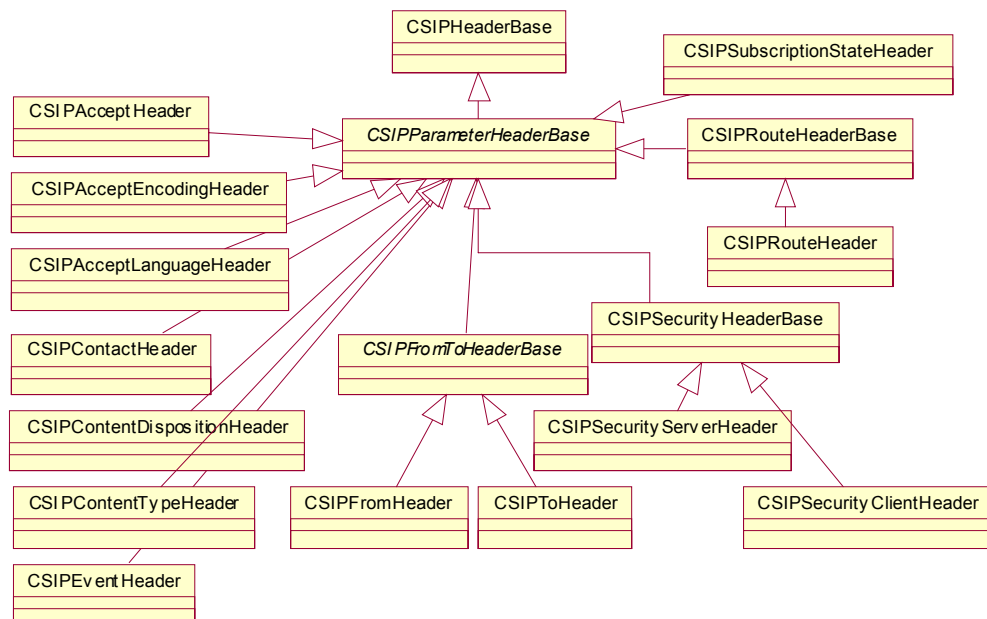


Figure 2: Relationship of SIP Codec classes (part two)

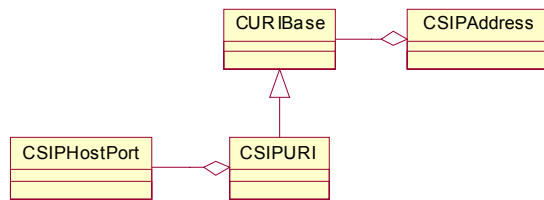


Figure 3: SIP Codec classes for handling SIP URI

The SIP Codec API defines the methods required to create the SIP headers of a SIP message.

`CSIPHeaderBase` is a base class for all the SIP headers. A client creates a new instance of a `CSIPXHeader` class, where “X” is the name of the header required, and sets the values of the header with the setter methods of this class.

Some header classes contain a SIP URL object, represented by its own class (Figure 3) in the SIP Codec API. See more detailed description from Chapter 3.

2.3 Usage

2.3.1 Protocol

The sequence diagram in Figure 4 illustrates the steps of creating a `CSIPContactHeader` instance.

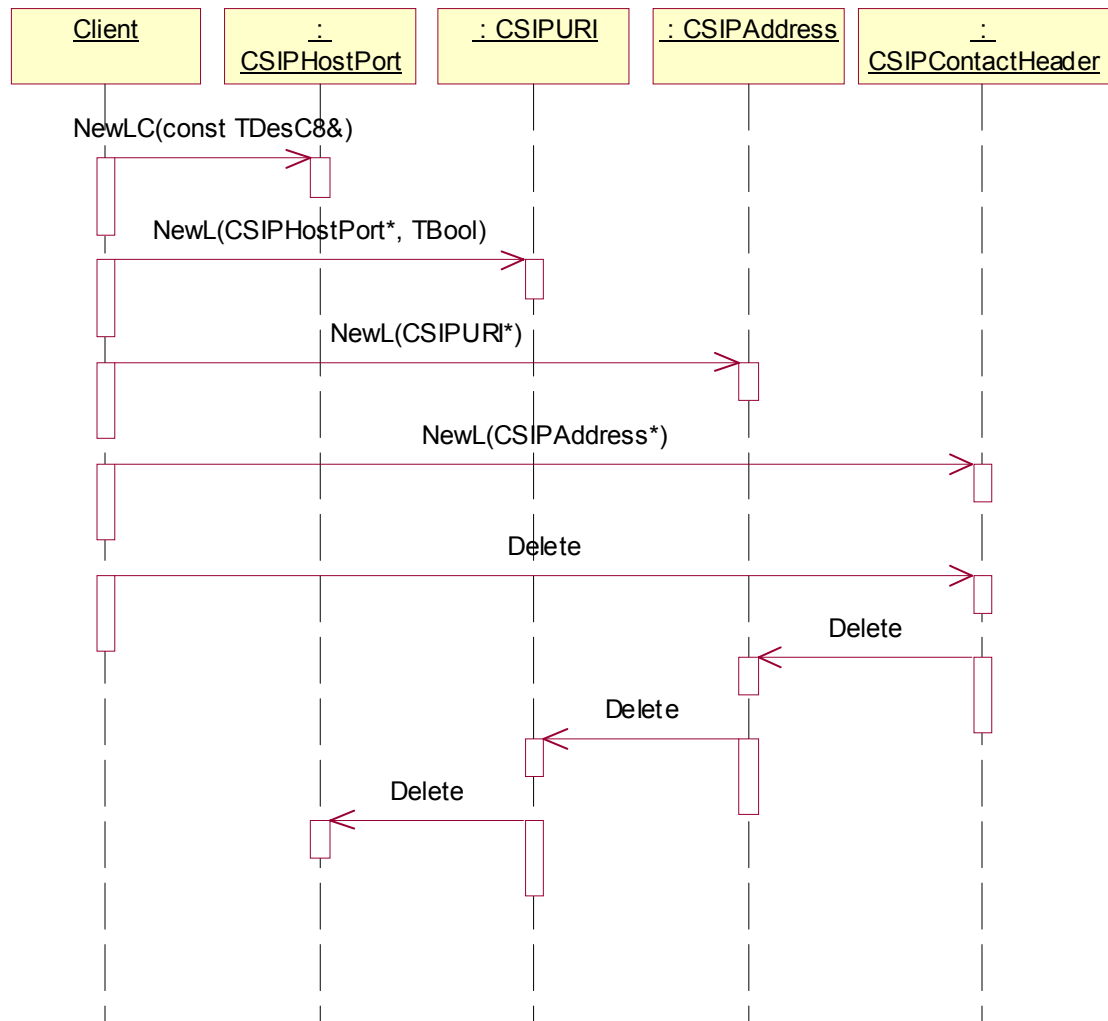


Figure 4: Creating a “Contact” header

2.3.2 Error handling

If an error occurs during the synchronous operation initiated by the client, the called function leaves with the proper error code.

2.3.3 Memory overhead

The amount of overhead memory used by every header object of the SIP Codec API is at least the same as the memory used by the header as text is using (class + size of the SIP headers).

2.3.4 Extensions to the API

There are no possible extensions to the SIP Codec API.

2.4 Example

This section presents pieces of code describing an implementation for a simple use of the API.

An example of how an application using SIP stack decodes a “Contact” header:

```
RPointerArray<CSIPHeaderBase> headers;
_LIT8 (KThreeParams, "sip:user@1.2.3.4;expires=0;q=1.0;p=value");
headers = CSIPContactHeader::DecodeL(KThreeParams);
TInt count = headers.Count(); //count == 1
iSIPContactHeader = static_cast<CSIPContactHeader*>(headers[0]);
headers.Reset();
iSIPContactHeader->HasExpiresParameter();//return ETrue
iSIPContactHeader->ExpiresParameter();//return zero
iSIPContactHeader->HasQParameter();//return ETrue
iSIPContactHeader->QParameter();//return 1.0
iSIPContactHeader->HasExtensionParam(_L8("p"));//return ETrue
iSIPContactHeader->ExtensionParamValue(_L8("p"));//return "value"
delete iSIPContactHeader; iSIPContactHeader = 0;
```

An example of how an application using SIP stack decodes several “Contact” headers:

```
RPointerArray<CSIPHeaderBase> headers;
_LIT8 (KHeaders1, "<sip:u1@host>, u2 <sip:u2@host> , sip:host ");
headers = CSIPContactHeader::DecodeL(KHeaders1);
headers.Count();//return 3
headers.ResetAndDestroy();
```

An example of how an application using SIP stack creates a CSIPContactHeader:

```
CSIPHostPort* hostPort = CSIPHostPort::NewLC();
CSIPURI* sipURI = CSIPURI::NewL(hostPort);
CleanupStack::Pop(); // hostPort, ownership given to sipURI
CleanupStack::PushL(sipURI);
CSIPAddress* sipAddress = CSIPAddress::NewL(sipURI);
CleanupStack::Pop(); // sipURI, ownership given to sipAddress
CleanupStack::PushL(sipAddress);
CSIPContactHeader* contactHeader = CSIPContactHeader::NewL(sipAddress);
CleanupStack::Pop(); // sipAddress, ownership given to contactHeader
```

An example of how an application using SIP stack decodes a “To” header:

```
RPointerArray<CSIPHeaderBase> headers;
_LIT8 (KTwoParams, "jane <sip:jane@aaaaa.com>;tag=xyz;p=1X");
headers = CSIPToHeader::DecodeL(KTwoParams);
headers.Count();//return 1
iHeader = static_cast<CSIPFromToHeaderBase*>(headers[0]);
headers.Reset();
iHeader->HasTag();//return ETrue
iHeader->Tag();//return "xyz"
iHeader->HasExtensionParam(_L8("p"));//return ETrue
iHeader->ExtensionParamValue(_L8("p"));//return "1X!"
```

2.5 Detailed Description

2.5.1 CSIPAcceptEncodingHeader

This class provides functions for setting and getting media types and parameters in the SIP “Accept-Encoding” header.

2.5.1.1 *Name*

Function	<code>static const TDesc8& Name ();</code>
Description	Gets the full name of the “Accept-Encoding” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesc8&</code> : The full name of the header
Postconditions	None
Exceptions	None

2.5.1.2 *DecodeL*

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesc8& aValue);</code>
Description	Constructs a CSIPAcceptEncodingHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesc8& aValue</code> : A value part of an “Accept-Encoding” header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> . An array containing one to many instances of <code>CSIPHeaderBase</code> . Can be downcast to <code>CSIPAcceptEncodingHeader</code> subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.3 *NewL*

Function	<code>static CSIPAcceptEncodingHeader* NewL (const TDesc8& aCodings);</code>
Description	Creates a new instance of CSIPAcceptEncodingHeader.
Preconditions	None
Parameter	<code>const TDesc8& aCodings</code> : The codings to set.
Return value	<code>CSIPAcceptEncodingHeader*</code> : A new instance of CSIPAcceptEncodingHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.4 *NewLC*

Function	<code>static CSIPAcceptEncodingHeader* NewLC (const TDesc8& aCodings);</code>
Description	Creates a new instance of CSIPAcceptEncodingHeader and puts it onto the cleanup stack.

Preconditions	None
Parameter	const TDesC8& aCodings: The codings to set.
Return value	CSIPAcceptEncodingHeader*: A new instance of CSIPAcceptEncodingHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.5 NewL

Function	static CSIPAcceptEncodingHeader* NewL (const CSIPAcceptEncodingHeader& aAcceptEncodingHeader);
Description	Creates a deep copy of the given “Accept-Encoding” header.
Preconditions	None
Parameter	const CSIPAcceptEncodingHeader& aAcceptEncodingHeader: “Accept-Encoding” header to be copied.
Return value	CSIPAcceptEncodingHeader*: A new instance of CSIPAcceptEncodingHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.6 NewLC

Function	static CSIPAcceptEncodingHeader* NewLC (const CSIPAcceptEncodingHeader& aAcceptEncodingHeader);
Description	Creates a deep copy of the given “Accept-Encoding” header and puts it onto the cleanup stack.
Preconditions	None
Parameter	const CSIPAcceptEncodingHeader& aAcceptEncodingHeader: “Accept-Encoding” header to be copied.
Return value	CSIPAcceptEncodingHeader*: A new instance of CSIPAcceptEncodingHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.7 CloneL

Function	CSIPHeaderBase* CloneL () const;
Description	Creates a deep copy of this CSIPAcceptEncodingHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.8 Codings

Function	<code>const TDesC8& Codings () const;</code>
Description	Gets the codings from the “Accept-Encoding” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : Codings.
Postconditions	None
Exceptions	None

2.5.1.9 SetCodingsL

Function	<code>void SetCodingsL (const TDesC8& aCodings);</code>
Description	Sets the codings in the “Accept-Encoding” header.
Preconditions	None
Parameter	<code>const TDesC8& aCodings</code> : The codings.
Return value	<code>void</code>
Postconditions	None
Exceptions	Function leaves with the <code>EErrorInAcceptEncodingHeader</code> error code if the token is empty or contains illegal characters.

2.5.1.10 HasQParameter

Function	<code>TBool HasQParameter () const;</code>
Description	Checks if the “q” parameter is present.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> if present, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.1.11 QParameter

Function	TReal QParameter () const;
Description	Gets the value of the “q” parameter if HasQParameter() == EFalse returns zero.
Preconditions	HasQParameter() == ETrue
Parameter	None
Return value	TReal: The “q” parameter value.
Postconditions	None
Exceptions	None

2.5.1.12 SetQParameterL

Function	void SetQParameterL (TReal aQValue);
Description	Sets the “q” parameter value.
Preconditions	None
Parameter	TReal aQValue: A “q” parameter value to set.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.13 MoreThanOneAllowed

Function	TBool MoreThanOneAllowed () const;
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, more than one instance of this header is allowed in a SIP message.
Postconditions	None
Exceptions	None

2.5.1.14 FullName

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “Accept-Encoding” header.
Preconditions	None
Parameter	None

Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.1.15 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPAcceptEncodingHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPAcceptEncodingHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.1.16 ~CSIPAcceptEncodingHeader

Function	~CSIPAcceptEncodingHeader
Description	Deletes the resources of CSIPAcceptEncodingHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.2 CSIPAcceptHeader

This class provides functions for setting and getting media types and parameters in the SIP “Accept” header.

2.5.2.1 Name

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Accept” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.

Postconditions	None
Exceptions	None

2.5.2.2 DecodeL

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesc8& aValue);</code>
Description	Constructs a CSIPAcceptHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesc8& aValue</code> : A value part of an "Accept" header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPAcceptHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.2.3 NewL

Function	<code>static CSIPAcceptHeader* NewL (const TDesc8& aMediaType, const TDesc8& aMediaSubtype);</code>
Description	Creates a new instance of CSIPAcceptHeader.
Preconditions	None
Parameter	<code>const TDesc8& aMediaType</code> : A media type to set. For example, "application" <code>const TDesc8& aMediaSubtype</code> : A media subtype to set. For example, "sdp"
Return value	<code>CSIPAcceptHeader*</code> : A new instance of CSIPAcceptHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.2.4 NewLC

Function	<code>static CSIPAcceptHeader* NewLC (const TDesc8& aMediaType, const TDesc8& aMediaSubtype);</code>
Description	Creates a new instance of CSIPAcceptHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesc8& aMediaType</code> : A media type to set. For example, "application" <code>const TDesc8& aMediaSubtype</code> : A media subtype to set. For example, "sdp"
Return value	<code>CSIPAcceptHeader*</code> : A new instance of CSIPAcceptHeader.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.2.5 NewL

Function	<code>static CSIPAcceptHeader* NewL (const CSIPAcceptHeader& aAcceptHeader);</code>
Description	Creates a deep copy of the given “Accept” header.
Preconditions	None
Parameter	<code>const CSIPAcceptHeader& aAcceptHeader</code> : “Accept” header to be copied.
Return value	<code>CSIPAcceptHeader*</code> : A new instance of <code>CSIPAcceptHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.2.6 NewLC

Function	<code>static CSIPAcceptHeader* NewLC (const CSIPAcceptHeader& aAcceptHeader);</code>
Description	Creates a deep copy of the given “Accept” header and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPAcceptHeader& aAcceptHeader</code> : “Accept” header to be copied.
Return value	<code>CSIPAcceptHeader*</code> : A new instance of <code>CSIPAcceptHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.2.7 CloneL

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this <code>CSIPAcceptHeader</code> object.
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.2.8 *IsEmpty*

Function	<code>TBool IsEmpty () const;</code>
Description	Checks if the header is empty. If the header is empty, the other getters will not return a meaningful value.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if empty, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.2.9 *MediaType*

Function	<code>const TDesc8& MediaType () const;</code>
Description	Gets the media type from the “Accept” header.
Preconditions	None
Parameter	None
Return value	const TDesc8&: A media type.
Postconditions	None
Exceptions	None

2.5.2.10 *SetMediaTypeL*

Function	<code>void SetMediaTypeL (const TDesc8& aMediaType);</code>
Description	Sets the media type in the “Accept” header.
Preconditions	<code>IsEmpty() == EFalse</code>
Parameter	const TDesc8& aMediaType: A media type.
Return value	void
Postconditions	None
Exceptions	The function leaves with the <code>KErrArgument</code> error code if <code>IsEmpty() == ETrue</code>. The function leaves with the <code>EErrorInAcceptHeader</code> error code if <code>aMediaType</code> is empty or contains illegal characters.

2.5.2.11 *MediaSubtype*

Function	<code>const TDesc8& MediaSubtype () const;</code>
Description	Gets the media subtype from the “Accept” header.
Preconditions	None

Parameter	None
Return value	const TDesC8&: A media subtype.
Postconditions	None
Exceptions	None

2.5.2.12 SetMediaSubtypeL

Function	void SetMediaSubtypeL (const TDesC8& aMediaSubtype);
Description	Sets the media subtype in the “Accept” header.
Preconditions	IsEmpty() == EFalse
Parameter	const TDesC8& aMediaSubtype: A media subtype.
Return value	void
Postconditions	None
Exceptions	The function leaves with the KErrArgument error code if IsEmpty() == ETrue. The function leaves with the EErrorInAcceptHeader error code if aMediaSubType is empty or contains illegal characters.

2.5.2.13 HasMediaParam

Function	TBool HasMediaParam (const TDesC8& aName) const;
Description	Checks if a media parameter is present.
Preconditions	None
Parameter	const TDesC8& aName: The name of the media parameter.
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.2.14 MediaParamValue

Function	const TDesC8& MediaParamValue (const TDesC8& aName) const;
Description	Gets a media parameter.
Preconditions	None
Parameter	const TDesC8& aName: The name of the media parameter.
Return value	const TDesC8&: The parameter value if present, otherwise a zero-length descriptor.
Postconditions	None
Exceptions	None

2.5.2.15 SetMediaParamL

Function	<code>void SetMediaParamL (const TDesC8& aName, const TDesC8& aValue);</code>
Description	Sets a media parameter in the “Accept” header.
Preconditions	None
Parameter	<code>const TDesC8& aName</code> : The parameter name to set. <code>const TDesC8& aValue</code> : The parameter value to set.
Return value	<code>void</code>
Postconditions	None
Exceptions	The function leaves with the <code>KErrArgument</code> error code if <code>IsEmpty() == ETrue</code> . The function leaves with the <code>EErrorInAcceptHeader</code> error code if <code>aName</code> or <code>aValue</code> are empty or contain illegal characters.

2.5.2.16 DeleteMediaParam

Function	<code>TInt DeleteMediaParam (const TDesC8& aName);</code>
Description	Deletes a media parameter.
Preconditions	None
Parameter	<code>const TDesC8& aName</code> : The parameter name to delete.
Return value	<code>TInt</code> : <code>KErrNone</code> if successfully deleted, <code>KErrNotFound</code> if the parameter was not found.
Postconditions	None
Exceptions	None

2.5.2.17 HasQParameter

Function	<code>TBool HasQParameter () const;</code>
Description	Checks if the “q” parameter is present.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> if present, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.2.18 QParameter

Function	TReal QParameter () const;
Description	Gets the value of the “q” parameter.
Preconditions	HasQParameter () == ETrue
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.2.19 SetQParameterL

Function	void SetQParameterL (TReal aQValue);
Description	Sets the “q” parameter value.
Preconditions	None
Parameter	TReal aQValue: A “q” parameter value to set.
Return value	void
Postconditions	None
Exceptions	The function leaves with the KErrArgument error code if IsEmpty () == ETrue. The function leaves with the EErrorInAcceptHeader error code if aQValue contain illegal characters.

2.5.2.20 MoreThanOneAllowed

Function	TBool MoreThanOneAllowed () const;
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, more than one instance of this header is allowed in a SIP message.
Postconditions	None
Exceptions	None

2.5.2.21 FullName

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “Accept” header.
Preconditions	None

Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.2.22 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPAcceptHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPAcceptHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.2.23 ~CSIPAcceptHeader

Function	~CSIPAcceptHeader
Description	Deletes the resources of CSIPAcceptHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.3 CSIPAcceptLanguageHeader

This is a class for SIP “Accept-Language” header manipulation. The class provides functions for setting and getting media types and parameters in the SIP “Accept-Language” header.

2.5.3.1 Name

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Accept-Language” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.

Postconditions	None
Exceptions	None

2.5.3.2 *DecodeL*

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);</code>
Description	Constructs a CSIPAcceptLanguageHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A value part of an "Accept-Language" header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> . An array containing one to many instances of <code>CSIPHeaderBase</code> . Can be downcast to <code>CSIPAcceptLanguageHeader</code> subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.3 *NewL*

Function	<code>static CSIPAcceptLanguageHeader* NewL (const TDesC8& aLanguageRange);</code>
Description	Creates a new instance of <code>CSIPAcceptLanguageHeader</code> .
Preconditions	None
Parameter	<code>const TDesC8& aLanguageRange</code> : A language range to set.
Return value	<code>CSIPAcceptLanguageHeader*</code> : A new instance of <code>CSIPAcceptLanguageHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.4 *NewLC*

Function	<code>static CSIPAcceptLanguageHeader* NewLC (const TDesC8& aLanguageRange);</code>
Description	Creates a new instance of <code>CSIPAcceptLanguageHeader</code> and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aLanguageRange</code> : A language range to set.
Return value	<code>CSIPAcceptLanguageHeader*</code> : A new instance of <code>CSIPAcceptLanguageHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.5 *NewL*

Function	<code>static CSIPAcceptLanguageHeader* NewL (const CSIPAcceptLanguageHeader& aAcceptLanguageHeader);</code>
Description	Creates a deep copy of the given “Accept-Language” header.
Preconditions	None
Parameter	<code>const CSIPAcceptLanguageHeader& aAcceptLanguageHeader</code> : “Accept-Language” header to be copied.
Return value	<code>CSIPAcceptLanguageHeader*</code> : A new instance of <code>CSIPAcceptLanguageHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.6 *NewLC*

Function	<code>static CSIPAcceptLanguageHeader* NewLC (const CSIPAcceptLanguageHeader& aAcceptLanguageHeader);</code>
Description	Creates a deep copy of the given “Accept-Language” header and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPAcceptLanguageHeader& aAcceptLanguageHeader</code> : “Accept-Language” header to be copied.
Return value	<code>CSIPAcceptLanguageHeader*</code> : A new instance of <code>CSIPAcceptLanguageHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.7 *CloneL*

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this <code>CSIPAcceptLanguageHeader</code> object.
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.8 *LanguageRange*

Function	<code>const TDesC8& LanguageRange () const;</code>
Description	Gets the language range from the “Accept-Language” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : A language range.
Postconditions	None
Exceptions	None

2.5.3.9 *SetLanguageRangeL*

Function	<code>void SetLanguageRangeL (const TDesC8& aLanguageRange);</code>
Description	Sets the language range in the “Accept-Language” header.
Preconditions	None
Parameter	<code>const TDesC8& aLanguageRange</code> : A language range.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.10 *HasQParameter*

Function	<code>TBool HasQParameter () const;</code>
Description	Checks if the “q” parameter is present.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> if present, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.3.11 *QParameter*

Function	<code>TReal QParameter () const;</code>
Description	Gets the value of the “q” parameter.
Preconditions	<code>HasQParameter() == ETrue</code>
Parameter	None
Return value	<code>TReal</code> : The “q” parameter value.

Postconditions	None
Exceptions	None

2.5.3.12 SetQParameterL

Function	void SetQParameterL (TReal aQValue);
Description	Sets the “q”parameter value.
Preconditions	None
Parameter	TReal aQValue: A “q” parameter value to set.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.13 MoreThanOneAllowed

Function	TBool MoreThanOneAllowed () const;
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, more than one instance of this header is allowed in a SIP message.
Postconditions	None
Exceptions	None

2.5.3.14 FullName

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “Accept-Language” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.3.15 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPAcceptLanguageHeader from RReadStream.

Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPAcceptLanguageHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.3.16 ~CSIPAcceptLanguageHeader

Function	~CSIPAcceptLanguageHeader
Description	Deletes the resources of CSIPAcceptLanguageHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.4 CSIPAddress

This is a class for SIP name-addr structure manipulation. The class provides functions for setting and getting parameters in the SIP name-addr structure.

2.5.4.1 DecodeL

Function	static CSIPAddress* DecodeL (const TDescC8& aValue);
Description	Constructs a CSIPAddress from the textual representation.
Preconditions	None
Parameter	const TDescC8& aValue: A SIP name-addr as text.
Return value	CSIPAddress*: A new instance of CSIPAddress.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.4.2 NewL

Function	static CSIPAddress* NewL (CSIPURI* aURI);
Description	Creates a new instance of CSIPAddress.
Preconditions	aURI != 0
Parameter	CSIPURI* aURI: A SIP(S)-URI. The ownership is transferred.
Return value	CSIPAddress*: A new instance of CSIPAddress.

Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with the KErrArgument error code if aURI == 0.

2.5.4.3 NewLC

Function	static CSIPAddress* NewLC (CSIPURI* aURI);
Description	Creates a new instance of CSIPAddress and puts it onto the cleanup stack.
Preconditions	aURI != 0
Parameter	CSIPURI* aURI: A SIP(S)-URI. The ownership is transferred.
Return value	CSIPAddress*: A new instance of CSIPAddress.
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with the KErrArgument error code if aURI == 0.

2.5.4.4 NewL

Function	static CSIPAddress* NewL (const TDesc8& aDisplayName, CSIPURI* aURI);
Description	Creates a new instance of CSIPAddress.
Preconditions	aURI != 0
Parameter	const TDesc8& aDisplayName: A SIP token or a SIP quoted-string CSIPURI* aURI: A SIP(S)-URI. The ownership is transferred.
Return value	CSIPAddress*: A new instance of CSIPAddress.
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with the KErrArgument error code if aURI == 0.

2.5.4.5 NewLC

Function	static CSIPAddress* NewLC (const TDesc8& aDisplayName, CSIPURI* aURI);
Description	Creates a new instance of CSIPAddress and puts it onto the cleanup stack.
Preconditions	aURI != 0
Parameter	const TDesc8& aDisplayName: A SIP token or a SIP quoted-string. CSIPURI* aURI: A SIP(S)-URI. The ownership is transferred.
Return value	CSIPAddress*: A new instance of CSIPAddress.
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with the KErrArgument error code if aURI == 0.

2.5.4.6 *NewL*

Function	<code>static CSIPAddress* NewL (const CSIPAddress& aSIPAddress);</code>
Description	Creates a deep copy of CSIPAddress.
Preconditions	None
Parameter	<code>const CSIPAddress& aSIPAddress</code> : The address to be copied.
Return value	<code>CSIPAddress*</code> : A new instance of CSIPAddress.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.4.7 *NewLC*

Function	<code>static CSIPAddress* NewLC (const CSIPAddress& aSIPAddress);</code>
Description	Creates a deep copy of CSIPAddress and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPAddress& aSIPAddress</code> : The address to be copied.
Return value	<code>CSIPAddress*</code> : A new instance of CSIPAddress.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.4.8 *operator==*

Function	<code>TBool operator==(const CSIPAddress& aSIPAddress) const;</code>
Description	Compares this object to another instance of CSIPAddress.
Preconditions	None
Parameter	<code>const CSIPAddress& aSIPAddress</code> : A CSIPAddress to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if the objects are equal, otherwise <code>EFa lse</code> .
Postconditions	None
Exceptions	None

2.5.4.9 *HasDisplayName*

Function	<code>TBool HasDisplayName () const;</code>
Description	Checks if this CSIPAddress has a display name.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> if the display name is present, otherwise <code>EFa lse</code> .

Postconditions	None
Exceptions	None

2.5.4.10 *DisplayName*

Function	<code>const TDesC8& DisplayName () const;</code>
Description	Gets the display name.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The display name if present, otherwise a zero-length descriptor.
Postconditions	None
Exceptions	None

2.5.4.11 *SetDisplayNameL*

Function	<code>void SetDisplayNameL (const TDesC8& aDisplayName);</code>
Description	Sets the display name.
Preconditions	None
Parameter	<code>const TDesC8& aDisplayName</code> : A SIP token or a SIP quoted-string.
Return value	<code>void</code>
Postconditions	None
Exceptions	The function leaves with the <code>EErrorInDisplayName</code> error code if <code>aDisplayName</code> is empty or contains illegal characters.

2.5.4.12 *URI*

Function	<code>const CSIPURI& URI () const;</code>
Description	Gets the URI part of the address as <code>const</code> .
Preconditions	None
Parameter	None
Return value	<code>const CSIPURI&</code> : A reference to the URI object.
Postconditions	None
Exceptions	None

2.5.4.13 URI

Function	CSIPURI& URI ();
Description	Gets the URI part of the address.
Preconditions	None
Parameter	None
Return value	const CSIPURI&: A reference to the URI object.
Postconditions	None
Exceptions	None

2.5.4.14 SetURIL

Function	void SetURIL (CSIPURI* aURI);
Description	Sets the URI part of the address.
Preconditions	aURI != 0
Parameter	CSIPURI* aURI: A pointer to the URI object. The ownership is transferred.
Return value	void
Postconditions	None
Exceptions	The function leaves with the KErrArgument error code if aURI == 0.

2.5.4.15 ToTextLC

Function	HBufC8* ToTextLC (TBool aUseAngleBrackets=EFALSE) const;
Description	Creates a textual representation and puts it onto the cleanup stack.
Preconditions	None
Parameter	TBool aUseAngleBrackets=EFALSE: If ETrue, sets the angle brackets.
Return value	HBufC8*: A textual representation of the object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.4.16 InternalizeL

Function	static CSIPAddress* InternalizeL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPAddress from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the externalized object.
Return value	CSIPAddress*: An instance of CSIPAddress.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.4.17 ExternalizeL

Function	void ExternalizeL (RWriteStream& aWriteStream);
Description	Writes the object to RwriteStream.
Preconditions	None
Parameter	RWriteStream& aWriteStream: A stream where the object is to be externalized.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.4.18 ~CSIPAddress();

Function	~CSIPAddress();
Description	Deletes the resources of CSIPAddress.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.5 CSIPAllowEventsHeader

This is a class for SIP “Allow-Events” header manipulation. The class provides functions for setting and getting parameters in the SIP “Allow-Events” header.

2.5.5.1 Name

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Allow-Events” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.5.2 *DecodeL*

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);</code>
Description	Constructs a CSIPAllowEventsHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A value part of an "Allow-Events" header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPAllowEventsHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.5.3 *NewL*

Function	<code>static CSIPAllowEventsHeader* NewL (const TDesC8& aEventPackage);</code>
Description	Creates a new instance of CSIPAllowEventsHeader.
Preconditions	None
Parameter	<code>const TDesC8& aEventPackage</code> : An Event-Package value.
Return value	<code>CSIPAllowEventsHeader*</code> : A new instance of CSIPAllowEventsHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.5.4 *NewL*

Function	<code>static CSIPAllowEventsHeader* NewL (const CSIPAllowEventsHeader& aAllowEventsHeader);</code>
Description	Creates a deep copy of CSIPAllowEventsHeader.
Preconditions	None
Parameter	<code>const CSIPAllowEventsHeader& aAllowEventsHeader</code> : "Allow-Events" header to be copied.
Return value	<code>CSIPAllowEventsHeader*</code> : A new instance of CSIPAllowEventsHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.5.5 *NewLC*

Function	<code>static CSIPAllowEventsHeader* NewLC (const TDesC8& aEventPackage);</code>
Description	Creates a new instance of CSIPAllowEventsHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aEventPackage</code> : An Event-Package value.
Return value	CSIPAllowEventsHeader*: A new instance of CSIPAllowEventsHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.5.6 *NewLC*

Function	<code>static CSIPAllowEventsHeader* NewLC (const CSIPAllowEventsHeader& aAllowEventsHeader);</code>
Description	Creates a deep copy of the given “Allow-Events” header and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPAllowEventsHeader& aAllowEventsHeader</code> : “Allow-Events” header to be copied.
Return value	CSIPAllowEventsHeader*: A new instance of CSIPAllowEventsHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.5.7 *CloneL*

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this CSIPAllowEventsHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.5.8 *FullName*

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “Allow-Events” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.5.9 *HasCompactName*

Function	TBool HasCompactName () const;
Description	Checks if this object has a compact form name.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, the object has a compact form name.
Postconditions	None
Exceptions	None

2.5.5.10 *CompactName*

Function	const TDesC8& CompactName () const;
Description	Gets the compact name of the “u” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The compact name of the “u” header.
Postconditions	None
Exceptions	None

2.5.5.11 *EventPackage*

Function	const TDesC8& EventPackage () const;
Description	Gets the Event-Package parameter from the “Allow-Events” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The Event-Package parameter.

Postconditions	None
Exceptions	None

2.5.5.12 *SetEventPackageL*

Function	<code>void SetEventPackageL (const TDesC8& aEventPackage);</code>
Description	Sets the Event-Package parameter in the “Allow-Events” header.
Preconditions	None
Parameter	<code>const TDesC8& aEventPackage</code> : An Event-Package parameter to set.
Return value	<code>void</code>
Postconditions	None
Exceptions	The function leaves with <code>EErrorInAllowEventsHeader</code> if <code>aEventPackage</code> contains illegal characters or doesn't contain a dot ('.').

2.5.5.13 *SetEventTemplatesL*

Function	<code>void SetEventTemplatesL (const MDesC8Array& aEventTemplates);</code>
Description	Sets the event templates overwriting all the existing ones.
Preconditions	None
Parameter	<code>const MDesC8Array& aEventTemplates</code> : The event templates.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.5.14 *EventTemplates*

Function	<code>const MDesC8Array& EventTemplates () const;</code>
Description	Gets the event templates.
Preconditions	None
Parameter	None
Return value	<code>const MDesC8Array&</code> : A reference to the currently set event templates.
Postconditions	None
Exceptions	None

2.5.5.15 *MoreThanOneAllowed*

Function	TBool MoreThanOneAllowed () const;
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, more than one instance of this header is allowed in a SIP message.
Postconditions	None
Exceptions	None

2.5.5.16 *ToTextValueL*

Function	HBufC8* ToTextValueL () const;
Description	Encodes the header's value into its textual representation.
Preconditions	None
Parameter	None
Return value	HBufC8*: A textual representation of the header's value. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.5.17 *InternalizeValueL*

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPAllowEventsHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPAllowEventsHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.5.18 *~CSIPAllowEventsHeader()*;

Function	~CSIPAllowEventsHeader ();
Description	Deletes the resources of CSIPAllowEventsHeader.
Preconditions	None
Parameter	None

Return value	None
Postconditions	None
Exceptions	None

2.5.6 CSIPAllowHeader

This is a base class for SIP “Allow” header manipulation. The class provides functions for setting and getting parameters in the SIP “Allow” header.

2.5.6.1 Name

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Allow” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.6.2 DecodeL

Function	static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);
Description	Constructs a CSIPAllowHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	const TDesC8& aValue: A value part of an “Allow” header.
Return value	RPointerArray<CSIPHeaderBase>: An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPAllowHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.3 NewL

Function	static CSIPAllowHeader* NewL (const TDesC8& aValue);
Description	Creates a new instance of CSIPAllowHeader.
Preconditions	None
Parameter	const TDesC8& aValue: A method value.
Return value	CSIPAllowHeader*: A new instance of CSIPAllowHeader.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.4 NewLC

Function	<code>static CSIPAllowHeader* NewLC (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPAllowHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A method value.
Return value	<code>CSIPAllowHeader*</code> : A new instance of CSIPAllowHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.5 NewL

Function	<code>static CSIPAllowHeader* NewL (const CSIPAllowHeader& aAllowHeader);</code>
Description	Creates a deep copy of the given CSIPAllowHeader instance.
Preconditions	None
Parameter	<code>const CSIPAllowHeader& aAllowHeader</code> : “Allow” header to be copied.
Return value	<code>CSIPAllowHeader*</code> : A new instance of CSIPAllowHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.6 NewLC

Function	<code>static CSIPAllowHeader* NewLC (const CSIPAllowHeader& aAllowHeader);</code>
Description	Creates a deep copy of the given CSIPAllowHeader instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPAllowHeader& aAllowHeader</code> : “Allow” header to be copied.
Return value	<code>CSIPAllowHeader*</code> : A new instance of CSIPAllowHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.7 *CloneL*

Function	CSIPHeaderBase* CloneL () const;
Description	Creates a deep copy of this CSIPAllowHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.8 *FullName*

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “Allow” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.6.9 *InternalizeValueL*

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPAllowHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPAllowHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.6.10 *~CSIPAllowHeader ()*;

Function	~CSIPAllowHeader ();
Description	Deletes the resources of CSIPAllowHeader.
Preconditions	None
Parameter	None
Return value	None

Postconditions	None
Exceptions	None

2.5.7 CSIPContactHeader

This is a Class for SIP “Contact” header manipulation. The class provides functions for setting and getting SIP “Contact” header fields.

2.5.7.1 Name

Function	<code>static const TDesC8& Name ();</code>
Description	Gets the full name of the “Contact” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The full name of the header.
Postconditions	None
Exceptions	None

2.5.7.2 DecodeL

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);</code>
Description	Constructs a CSIPContactHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A value part of a “Contact” header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPContactHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.3 NewL

Function	<code>static CSIPContactHeader* NewL ();</code>
Description	Creates a new instance of CSIPContactHeader with the address set as a STAR. The setters may not be used after this.
Preconditions	None
Parameter	None
Return value	<code>CSIPContactHeader*</code> : A new instance of CSIPContactHeader.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.4 NewLC

Function	<code>static CSIPContactHeader* NewLC ();</code>
Description	Creates a new instance of CSIPContactHeader with the address set as a STAR and puts it onto the cleanup stack. The setters may not be used after this.
Preconditions	None
Parameter	None
Return value	CSIPContactHeader*: A new instance of CSIPContactHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.5 NewL

Function	<code>static CSIPContactHeader* NewL (CSIPAddress* aSIPAddress);</code>
Description	Creates a new instance of CSIPContactHeader.
Preconditions	<code>aSIPAddress != 0</code>
Parameter	CSIPAddress* aSIPAddress: A name-address. The ownership is transferred.
Return value	CSIPContactHeader*: A new instance of CSIPContactHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.6 NewLC

Function	<code>static CSIPContactHeader* NewLC (CSIPAddress* aSIPAddress);</code>
Description	Creates a new instance of CSIPContactHeader and puts it onto the cleanup stack.
Preconditions	<code>aSIPAddress != 0</code>
Parameter	CSIPAddress* aSIPAddress: A name-address. The ownership is transferred.
Return value	CSIPContactHeader*: A new instance of CSIPContactHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.7 *NewL*

Function	<code>static CSIPContactHeader* NewL (const CSIPContactHeader& aContactHeader);</code>
Description	Creates a deep copy of the given CSIPContactHeader instance.
Preconditions	None
Parameter	<code>const CSIPContactHeader& aContactHeader</code> : “Contact” header to be copied.
Return value	<code>CSIPContactHeader*</code> : A new instance of CSIPContactHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.8 *NewLC*

Function	<code>static CSIPContactHeader* NewLC (const CSIPContactHeader& aContactHeader);</code>
Description	Creates a deep copy of the given CSIPContactHeader instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPContactHeader& aContactHeader</code> : “Contact” header to be copied.
Return value	<code>CSIPContactHeader*</code> : A new instance of CSIPContactHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.9 *CloneL*

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this CSIPContactHeader object.
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.10 *operator==*

Function	<code>TBool operator==(const CSIPContactHeader& aContactHeader) const;</code>
Description	Compares this object to another instance of CSIPContactHeader.
Preconditions	None

Parameter	const CSIPContactHeader& aContactHeader: A CSIPContactHeader to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	

2.5.7.11 Star

Function	TBool Star () const;
Description	Checks if the address is set to a STAR (*).
Preconditions	None
Parameter	None
Return value	TBool: ETrue if the address is a *, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.7.12 HasExpiresParameter

Function	TBool HasExpiresParameter () const;
Description	Checks if the “expires” parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.7.13 ExpiresParameter

Function	TUint ExpiresParameter () const;
Description	Gets the value of the “expires” parameter.
Preconditions	HasExpiresParameter() == ETrue Star() == EFalse
Parameter	None
Return value	TUint: The “expires” parameter. Returns zero if the preconditions are not met.
Postconditions	None
Exceptions	None

2.5.7.14 *SetExpiresParameterL*

Function	void SetExpiresParameterL (TUint aExpiresParam);
Description	Sets the “expires” parameter in the “Contact” header.
Preconditions	None
Parameter	TUint aExpiresParam: An “expires” parameter value to set.
Return value	void
Postconditions	None
Exceptions	The function leaves with EErrorContactSetToStar if IsStar() == ETrue.

2.5.7.15 *HasQParameter*

Function	TBool HasQParameter () const;
Description	Checks if the “q” parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	

2.5.7.16 *QParameter*

Function	TReal QParameter () const;
Description	Gets the value of the “q” parameter.
Preconditions	HasQParameter() == ETrue Star() == EFalse
Parameter	None
Return value	TReal: The “q” parameter value. Returns zero if the preconditions are not met.
Postconditions	None
Exceptions	None

2.5.7.17 *SetQParameterL*

Function	void SetQParameterL (TReal aQValue);
Description	Sets the “q” parameter value in the “Contact” header.
Preconditions	Star() == EFalse
Parameter	TReal aQValue: A “q” parameter value to set.
Return value	void

Postconditions	None
Exceptions	The function leaves with EErrorContactSetToStar if IsStar() == ETrue.

2.5.7.18 SIPAddress

Function	const CSIPAddress* SIPAddress () const;
Description	Gets the name-address as const.
Preconditions	None
Parameter	None
Return value	const CSIPAddress*: A name-address or 0 if not present. The ownership is NOT transferred.
Postconditions	None
Exceptions	None

2.5.7.19 SIPAddress

Function	CSIPAddress* SIPAddress ();
Description	Gets the name-address.
Preconditions	None
Parameter	None
Return value	CSIPAddress*: A name-address or 0 if not present. The ownership is NOT transferred.
Postconditions	None
Exceptions	None

2.5.7.20 SetSIPAddressL

Function	void SetSIPAddressL (CSIPAddress* aSIPAddress);
Description	Sets the name-address.
Preconditions	Star() == EFalse aSIPAddress != 0
Parameter	CSIPAddress* aSIPAddress: A name-address to set. The ownership is transferred.
Return value	void
Postconditions	None
Exceptions	The function leaves with EErrorContactSetToStar if IsStar() == ETrue. The function leaves with EErrorInContactAddress if aSIPAddress == 0.

2.5.7.21 *MoreThanOneAllowed*

Function	TBool MoreThanOneAllowed () const;
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, more than one instance of this header is allowed in a SIP message.
Postconditions	None
Exceptions	None

2.5.7.22 *FullName*

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “Contact” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.7.23 *HasCompactName*

Function	TBool HasCompactName () const;
Description	Checks if this object has a compact form name.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, the object has a compact form name.
Postconditions	None
Exceptions	None

2.5.7.24 *CompactName*

Function	const TDesC8& CompactName () const;
Description	Gets the compact name of the header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The compact name of the “m” header.

Postconditions	None
Exceptions	None

2.5.7.25 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPContactHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPContactHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.7.26 ~CSIPContactHeader ();

Function	~CSIPContactHeader ();
Description	Deletes the resources of CSIPContactHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.8 CSIPContentDispositionHeader

This is a class for SIP “Content-Disposition” header manipulation. The class provides functions for setting and getting parameters in the SIP “Content-Disposition” header.

2.5.8.1 Name

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Content-Disposition” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.8.2 *DecodeL*

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);</code>
Description	Constructs a CSIPContentDispositionHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A value part of a "Content-Disposition" header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing exactly one instance of CSIPHeaderBase. Can be downcast to CSIPContentDispositionHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.3 *NewL*

Function	<code>static CSIPContentDispositionHeader* NewL (const TDesC8& aDispType);</code>
Description	Creates a new instance of CSIPContentDispositionHeader.
Preconditions	None
Parameter	<code>const TDesC8& aDispType</code> : A Disp-Type value.
Return value	<code>CSIPContentDispositionHeader*</code> : A new instance of CSIPContentDispositionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.4 *NewL*

Function	<code>static CSIPContentDispositionHeader* NewL (const TDesC8& aDispType, const TDesC8& aHandlingParam);</code>
Description	Creates a new instance of CSIPContentDispositionHeader.
Preconditions	None
Parameter	<code>const TDesC8& aDispType</code> : A Disp-Type value. <code>const TDesC8& aHandlingParam</code> : A handling value.
Return value	<code>CSIPContentDispositionHeader*</code> : A new instance of CSIPContentDispositionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.5 *NewL*

Function	<code>static CSIPContentDispositionHeader* NewL (const CSIPContentDispositionHeader& aContentDispositionHeader);</code>
Description	Creates a deep copy of CSIPContentDispositionHeader.
Preconditions	None
Parameter	<code>const CSIPContentDispositionHeader& aContentDispositionHeader:</code> “Content-Disposition” header to be copied.
Return value	CSIPContentDispositionHeader*: A new instance of CSIPContentDispositionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.6 *NewLC*

Function	<code>static CSIPContentDispositionHeader* NewLC (const TDesC8& aDispType);</code>
Description	Creates a new instance of CSIPContentDispositionHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aDispType:</code> A Disp-Type value.
Return value	CSIPContentDispositionHeader*: A new instance of CSIPContentDispositionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.7 *NewLC*

Function	<code>static CSIPContentDispositionHeader* NewLC (const TDesC8& aDispType, const TDesC8& aHandlingParam);</code>
Description	Creates a new instance of CSIPContentDispositionHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aDispType:</code> A Disp-Type value. <code>const TDesC8& aHandlingParam:</code> A handling value.
Return value	CSIPContentDispositionHeader*: A new instance of CSIPContentDispositionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.8 *NewLC*

Function	<code>static CSIPContentDispositionHeader* NewLC (const CSIPContentDispositionHeader& aContentDispositionHeader);</code>
Description	Creates a deep copy of a CSIPContentDispositionHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPContentDispositionHeader& aContentDispositionHeader:</code> “Content-Disposition” header to be copied.
Return value	<code>CSIPContentDispositionHeader*</code> : A new instance of CSIPContentDispositionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.9 *CloneL*

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this CSIPContentDispositionHeader object.
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.10 *FullName*

Function	<code>const TDesC8& FullName () const;</code>
Description	Gets the full name of the “Content-Disposition” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The full name of the header.
Postconditions	None
Exceptions	None

2.5.8.11 *DispType*

Function	<code>const TDesC8& DispType () const;</code>
Description	Gets the disp-type parameter from the “Content-Disposition” header.
Preconditions	None

Parameter	None
Return value	const TDesC8&: The disp-type parameter.
Postconditions	None
Exceptions	None

2.5.8.12 SetDispTypeL

Function	void SetDispTypeL (const TDesC8& aDispType);
Description	Sets the disp-type parameter in the “Content-Disposition” header.
Preconditions	None
Parameter	const TDesC8& aDispType: A disp-type parameter to set.
Return value	void
Postconditions	None
Exceptions	The function leaves with the EErrorInContentDispositionHeader error code if aDispType contains illegal characters.

2.5.8.13 HasHandlingParam

Function	TBool HasHandlingParam () const;
Description	Checks if the handling parameter is present in the “Content-Disposition” header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.8.14 HandlingParam

Function	const TDesC8& HandlingParam () const;
Description	Gets the handling parameter from the “Content-Disposition” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The handling parameter.
Postconditions	None
Exceptions	None

2.5.8.15 SetHandlingParamL

Function	<code>void SetHandlingParamL (const TDesC8& aHandlingParam);</code>
Description	Sets the handling parameter in the “Content-Disposition” header.
Preconditions	None
Parameter	<code>const TDesC8& aHandlingParam</code> : A handling parameter to set.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.16 InternalizeValueL

Function	<code>static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);</code>
Description	Constructs an instance of <code>CSIPContentDispositionHeader</code> from <code>RReadStream</code> .
Preconditions	None
Parameter	<code>RReadStream& aReadStream</code> : A stream containing the value of the externalized header object (header name not included).
Return value	<code>CSIPHeaderBase*</code> : An instance of <code>CSIPContentDispositionHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.8.17 ~CSIPContentDispositionHeader ();

Function	<code>~CSIPContentDispositionHeader ();</code>
Description	Deletes the resources of <code>CSIPContentDispositionHeader</code> .
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.9 CSIPContentEncodingHeader

This is a base class for SIP “Content-Encoding” header manipulation. The class provides functions for setting and getting parameters in the SIP “Content-Encoding” header.

2.5.9.1 *Name*

Function	<code>static const TDesC8& Name ();</code>
Description	Gets the full name of the “Content-Encoding” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The full name of the header.
Postconditions	None
Exceptions	None

2.5.9.2 *DecodeL*

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);</code>
Description	Constructs a CSIPContentEncodingHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A value part of a “Content-Encoding” header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPContentEncodingHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.3 *NewL*

Function	<code>static CSIPContentEncodingHeader* NewL (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPContentEncodingHeader.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A content-coding value.
Return value	<code>CSIPContentEncodingHeader*</code> : A new instance of CSIPContentEncodingHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.4 *NewLC*

Function	<code>static CSIPContentEncodingHeader* NewLC (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPContentEncodingHeader.
Preconditions	None

Parameter	<code>const TDesC8& aValue</code> : A content-coding value.
Return value	<code>CSIPContentEncodingHeader*</code> : A new instance of <code>CSIPContentEncodingHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.5 *NewL*

Function	<code>static CSIPContentEncodingHeader* NewL (const CSIPContentEncodingHeader& aContentEncodingHeader);</code>
Description	Creates a deep copy of the given <code>CSIPContentEncodingHeader</code> instance.
Preconditions	None
Parameter	<code>const CSIPContentEncodingHeader& aContentEncodingHeader</code> : Parameter <code>aContentEncodingHeader</code> to be copied.
Return value	<code>CSIPContentEncodingHeader*</code> : A new instance of <code>CSIPContentEncodingHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.6 *NewLC*

Function	<code>static CSIPContentEncodingHeader* NewLC (const CSIPContentEncodingHeader& aContentEncodingHeader);</code>
Description	Creates a deep copy of the given <code>CSIPContentEncodingHeader</code> instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPContentEncodingHeader& aContentEncodingHeader</code> : Parameter <code>aContentEncodingHeader</code> to be copied.
Return value	<code>CSIPContentEncodingHeader*</code> : A new instance of <code>CSIPContentEncodingHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.7 *CloneL*

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this <code>CSIPContentEncodingHeader</code> object.
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.8 *FullName*

Function	<code>const TDesc8& FullName () const;</code>
Description	Gets the full name of the “Content-Encoding” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesc8&</code> : The full name of the header.
Postconditions	None
Exceptions	None

2.5.9.9 *HasCompactName*

Function	<code>TBool HasCompactName () const;</code>
Description	Checks if this object has a compact form name.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> , the object has a compact form name.
Postconditions	None
Exceptions	None

2.5.9.10 *CompactName*

Function	<code>const TDesc8& CompactName () const;</code>
Description	Gets the compact name of the header.
Preconditions	None
Parameter	None
Return value	<code>const TDesc8&</code> : The compact name of the “e” header.
Postconditions	None
Exceptions	None

2.5.9.11 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPContentEncodingHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPContentEncodingHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.9.12 ~CSIPContentEncodingHeader ();

Function	~CSIPContentEncodingHeader ();
Description	Deletes the resources of CSIPContentEncodingHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.10 CSIPContentTypeHeader

This is a class for SIP “Content-Type” header manipulation. The class provides functions for setting and getting media types and parameters in the SIP “Content-Type” header.

2.5.10.1 Name

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Content-Type” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.10.2 DecodeL

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesc8& aValue);</code>
Description	Constructs a CSIPContentTypeHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesc8& aValue</code> : A value part of a "Content-Type" header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing exactly one instance of CSIPHeaderBase. Can be downcast to CSIPContentTypeHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.3 NewL

Function	<code>static CSIPContentTypeHeader* NewL (const TDesc8& aMediaType, const TDesc8& aMediaSubtype);</code>
Description	Creates a new instance of CSIPContentTypeHeader.
Preconditions	None
Parameter	<code>const TDesc8& aMediaType</code> : A media type to set. For example, "application" <code>const TDesc8& aMediaSubtype</code> : A media subtype to set. For example, "sdp"
Return value	<code>CSIPContentTypeHeader*</code> : A new instance of CSIPContentTypeHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.4 NewLC

Function	<code>static CSIPContentTypeHeader* NewLC (const TDesc8& aMediaType, const TDesc8& aMediaSubtype);</code>
Description	Creates a new instance of CSIPContentTypeHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesc8& aMediaType</code> : A media type to set. For example, "application" <code>const TDesc8& aMediaSubtype</code> : A media subtype to set. For example, "sdp"
Return value	<code>CSIPContentTypeHeader*</code> : A new instance of CSIPContentTypeHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.5 NewL

Function	<code>static CSIPContentTypeHeader* NewL (const CSIPContentTypeHeader& aContentTypeHeader);</code>
Description	Creates a deep copy of the given “Content-Type” header.
Preconditions	None
Parameter	<code>const CSIPContentTypeHeader& aContentTypeHeader</code> : Parameter <code>aContentTypeHeader</code> to be copied.
Return value	<code>CSIPContentTypeHeader*</code> : A new instance of <code>CSIPContentTypeHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.6 NewLC

Function	<code>static CSIPContentTypeHeader* NewLC (const CSIPContentTypeHeader& aContentTypeHeader);</code>
Description	Creates a deep copy of the given “Content-Type” header and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPContentTypeHeader& aContentTypeHeader</code> : Parameter <code>aContentTypeHeader</code> to be copied.
Return value	<code>CSIPContentTypeHeader*</code> : A new instance of <code>CSIPContentTypeHeader</code> .
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.7 CloneL

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this <code>CSIPContentTypeHeader</code> object.
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.8 MediaType

Function	const TDesC8& MediaType () const;
Description	Gets the media type from the “Content-Type” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: A media type.
Postconditions	None
Exceptions	None

2.5.10.9 SetMediaTypeL

Function	void SetMediaTypeL (const TDesC8& aMediaType);
Description	Sets the media type in the “Content-Type” header.
Preconditions	None
Parameter	const TDesC8& aMediaType: A media type.
Return value	void
Postconditions	None
Exceptions	The function leaves with the EErrorInContentMediaType error code if aMediaType is empty or contains illegal characters.

2.5.10.10 MediaSubtype

Function	const TDesC8& MediaSubtype () const;
Description	Gets the media subtype from the “Content-Type” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: A media subtype.
Postconditions	None
Exceptions	None

2.5.10.11 SetMediaSubtypeL

Function	void SetMediaSubtypeL (const TDesC8& aMediaSubtype);
Description	Sets the media subtype in the “Content-Type” header.
Preconditions	None
Parameter	const TDesC8& aMediaSubtype: A media subtype.
Return value	void

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.12 *MoreThanOneAllowed*

Function	TBool MoreThanOneAllowed () const;
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: EFalse, no more than one instance of this header is allowed in a SIP message.
Postconditions	None
Exceptions	None

2.5.10.13 *FullName*

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “Content-Type” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.10.14 *HasCompactName*

Function	TBool HasCompactName () const;
Description	Checks if this object has a compact form name.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, the object has a compact form name.
Postconditions	None
Exceptions	None

2.5.10.15 *CompactName*

Function	const TDesC8& CompactName () const;
Description	Gets the compact name of the “c” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The compact name of the “c” header.
Postconditions	None
Exceptions	None

2.5.10.16 *InternalizeValueL*

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPContentTypeHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPContentTypeHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.10.17 *~CSIPContentTypeHeader ()*;

Function	~CSIPContentTypeHeader ();
Description	Deletes the resources of CSIPContentTypeHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.11 CSIPSeqHeader

This is a class for SIP “CSeq” header manipulation. The class provides functions for setting and getting sequence number and method in the SIP “CSeq” header.

2.5.11.1 Name

Function	static const TDesc8& Name ();
Description	Gets the full name of the “CSeq” header.
Preconditions	None
Parameter	None
Return value	const TDesc8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.11.2 DecodeL

Function	static RPointerArray<CSIPHeaderBase> DecodeL (const TDesc8& aValue);
Description	Constructs a CSIPCSeqHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	const TDesc8& aValue: A value part of a “CSeq” header (for example, “1 REGISTER”).
Return value	RPointerArray<CSIPHeaderBase>: An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPCSeqHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.3 NewL

Function	static CSIPCSeqHeader* NewL (TUint aSeq, const TDesc8& aMethod);
Description	Creates a new instance of CSIPCSeqHeader.
Preconditions	None
Parameter	TUint aSeq: A sequence number to set. const TDesc8& aMethod: A method to set. For example, “REGISTER”
Return value	CSIPCSeqHeader*: A new instance of CSIPCSeqHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.4 NewLC

Function	static CSIPSeqHeader* NewLC (TUint aSeq, const TDesC8& aMethod);
Description	Creates a new instance of CSIPSeqHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	TUint aSeq: A sequence number to set. const TDesC8& aMethod: A method to set. For example, "REGISTER"
Return value	CSIPSeqHeader*: A new instance of CSIPSeqHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.5 NewL

Function	static CSIPSeqHeader* NewL (const CSIPSeqHeader& aCSeqHeader);
Description	Creates a deep copy of CSIPSeqHeader.
Preconditions	None
Parameter	const CSIPSeqHeader& aCSeqHeader: CSIPSeqHeader to be copied.
Return value	CSIPSeqHeader*: A new instance of CSIPSeqHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.6 NewLC

Function	static CSIPSeqHeader* NewLC (const CSIPSeqHeader& aCSeqHeader);
Description	Creates a deep copy of CSIPSeqHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	const CSIPSeqHeader& aCSeqHeader: CSIPSeqHeader to be copied.
Return value	CSIPSeqHeader*: A new instance of CSIPSeqHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.7 CloneL

Function	CSIPHeaderBase* CloneL () const;
Description	Creates a deep copy of this CSIPSeqHeader object.
Preconditions	None

Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.8 operator==

Function	TBool operator==(const CSIPSeqHeader& aSeqHeader);
Description	Compares this object to another instance of CSIPSeqHeader.
Preconditions	None
Parameter	const CSIPSeqHeader& aSeqHeader: A CSIPSeqHeader to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.11.9 Seq

Function	TUint Seq () const;
Description	Gets the sequence number from the “CSeq” header.
Preconditions	None
Parameter	None
Return value	TUint: The current sequence number.
Postconditions	None
Exceptions	None

2.5.11.10 SetSeq

Function	void SetSeq (TUint aSeq);
Description	Sets the sequence number in the “CSeq” header.
Preconditions	None
Parameter	TUint aSeq: A sequence number to set.
Return value	void
Postconditions	None
Exceptions	None

2.5.11.11 *Method*

Function	<code>const TDesC8& Method () const;</code>
Description	Gets the method from the “CSeq” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The method.
Postconditions	None
Exceptions	None

2.5.11.12 *SetMethodL*

Function	<code>void SetMethodL (const TDesC8& aMethod);</code>
Description	Sets the method in the “CSeq” header.
Preconditions	None
Parameter	<code>const TDesC8& aMethod</code> : A method to set.
Return value	<code>void</code>
Postconditions	None
Exceptions	The function leaves with the <code>EErrorInCSeqMethod</code> error code if <code>aMethod</code> is empty or contains illegal characters.

2.5.11.13 *FullName*

Function	<code>const TDesC8& FullName () const;</code>
Description	Gets the full name of the “CSeq” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The full name of the header.
Postconditions	None
Exceptions	None

2.5.11.14 *ToTextValueL*

Function	<code>HBufC8* ToTextValueL () const;</code>
Description	Encodes the header's value into its textual representation.
Preconditions	None
Parameter	None
Return value	<code>HBufC8*</code> : A textual representation of the header's value. The ownership is transferred.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.15 *InternalizeValueL*

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPCSeqHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPCSeqHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.11.16 *~CSIPCSeqHeader ()*;

Function	~CSIPCSeqHeader ();
Description	Deletes the resources of CSIPCSeqHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.12 CSIPEventHeader

This is a class for SIP “Event” header manipulation. The class provides functions for setting and getting parameters in the SIP “Event” header.

2.5.12.1 *Name*

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Event” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.12.2 DecodeL

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);</code>
Description	Constructs a CSIPEventHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A value part of an "Event" header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> . An array containing exactly one instance of CSIPHeaderBase. Can be downcast to CSIPEventHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.12.3 NewL

Function	<code>static CSIPEventHeader* NewL (const TDesC8& aEventPackage);</code>
Description	Creates a new instance of CSIPEventHeader.
Preconditions	None
Parameter	<code>const TDesC8& aEventPackage</code> : An Event-Package value.
Return value	<code>CSIPEventHeader*</code> : A new instance of CSIPEventHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.12.4 NewL

Function	<code>static CSIPEventHeader* NewL (const CSIPEventHeader& aEventHeader);</code>
Description	Creates a deep copy of CSIPEventHeader.
Preconditions	None
Parameter	<code>const CSIPEventHeader& aEventHeader</code> : CSIPEventHeader to be copied.
Return value	<code>CSIPEventHeader*</code> : A new instance of CSIPEventHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.12.5 NewLC

Function	<code>static CSIPEventHeader* NewLC (const TDesC8& aEventPackage);</code>
Description	Creates a new instance of CSIPEventHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aEventPackage</code> : An Event-Package value.
Return value	<code>CSIPEventHeader*</code> : A new instance of CSIPEventHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.12.6 NewLC

Function	<code>static CSIPEventHeader* NewLC (const CSIPEventHeader& aEventHeader);</code>
Description	Creates a deep copy of CSIPEventHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPEventHeader& aEventHeader</code> : CSIPEventHeader to be copied.
Return value	<code>CSIPEventHeader*</code> : A new instance of CSIPEventHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.12.7 CloneL

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this CSIPEventHeader object.
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.12.8 operator==

Function	<code>TBool operator==(const CSIPEventHeader& aEventHeader) const;</code>
Description	Compares this instance to another CSIPEventHeader instance.
Preconditions	None
Parameter	<code>const CSIPEventHeader& aEventHeader</code> : A header to compare to.

Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.12.9 *FullName*

Function	const TDesc8& FullName () const;
Description	Gets the full name of the “Event” header.
Preconditions	None
Parameter	None
Return value	const TDesc8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.12.10 *HasCompactName*

Function	TBool HasCompactName () const;
Description	Checks if this object has a compact form name.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, the object has a compact form name.
Postconditions	None
Exceptions	None

2.5.12.11 *CompactName*

Function	const TDesc8& CompactName () const;
Description	Gets the compact name of the “o” header.
Preconditions	None
Parameter	None
Return value	const TDesc8&: The compact name of the “o” header.
Postconditions	None
Exceptions	None

2.5.12.12 *EventPackage*

Function	<code>const TDesC8& EventPackage () const;</code>
Description	Gets the Event-Package parameter from the “Event” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The Event-Package parameter.
Postconditions	None
Exceptions	None

2.5.12.13 *SetEventPackageL*

Function	<code>void SetEventPackageL (const TDesC8& aEventPackage);</code>
Description	Sets the Event-Package parameter in the “Event” header.
Preconditions	None
Parameter	<code>const TDesC8& aEventPackage</code> : An Event-Package parameter to set.
Return value	<code>void</code>
Postconditions	None
Exceptions	The function leaves with the <code>EErrorInEventHeader</code> error code if <code>aEventPackage</code> contains illegal characters or doesn't contain a dot ('.').

2.5.12.14 *SetEventTemplatesL*

Function	<code>void SetEventTemplatesL (const MDesC8Array& aEventTemplates);</code>
Description	Sets the event templates overwriting all the existing ones.
Preconditions	None
Parameter	<code>const MDesC8Array& aEventTemplates</code> : The event templates.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.12.15 *EventTemplates*

Function	<code>const MDesC8Array& EventTemplates () const;</code>
Description	Gets the event templates.
Preconditions	None
Parameter	None
Return value	<code>const MDesC8Array&</code> : A reference to the currently set event templates.

Postconditions	None
Exceptions	None

2.5.12.16 *HasIdParam*

Function	TBool HasIdParam () const;
Description	Checks if the “id” parameter is present in the “Event” header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.12.17 *IdParam*

Function	const TDesc8& IdParam () const;
Description	Gets the “id” parameter from the “Event” header.
Preconditions	None
Parameter	None
Return value	const TDesc8&: The “id” parameter if present, otherwise a zero-length descriptor.
Postconditions	None
Exceptions	None

2.5.12.18 *SetIdParamL*

Function	void SetIdParamL (const TDesc8& aIdParam);
Description	Sets the “id” parameter in the “Event” header.
Preconditions	None
Parameter	const TDesc8& aIdParam: An “id” parameter to set.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.12.19 *InternalizeValueL*

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPEventHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPEventHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.12.20 *~CSIPEventHeader ()*;

Function	~CSIPEventHeader ();
Description	Deletes the resources of CSIPEventHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.13 CSIPExpiresHeader

This is a class for SIP “Expires” header manipulation. The class encapsulates an “Expires” header value.

2.5.13.1 *Name*

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Expires” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.13.2 *DecodeL*

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);</code>
Description	Constructs a CSIPExpiresHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A value part of an "Expires" header (for example, "3600").
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing exactly one instance of <code>CSIPHeaderBase</code> . Can be downcast to <code>CSIPExpiresHeader</code> subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.3 *CSIPExpiresHeader*

Function	<code>CSIPExpiresHeader (TUint aValue);</code>
Description	Constructor
Preconditions	None
Parameter	<code>TUint aValue</code> : The value to set.
Return value	None
Postconditions	None
Exceptions	None

2.5.13.4 *CloneL*

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this <code>CSIPExpiresHeader</code> object.
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.5 *FullName*

Function	<code>const TDesC8& FullName () const;</code>
Description	Gets the full name of the "Expires" header.
Preconditions	None

Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.13.6 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPExpiresHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPExpiresHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.13.7 ~CSIPExpiresHeader ();

Function	~CSIPExpiresHeader ();
Description	Deletes the resources of CSIPExpiresHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.14 CSIPExtensionHeader

This is a class for a SIP extension header. The class stores unknown and extension headers that are either not supported by the current SIP Codec implementation or not specified in the current SIP RFC or both.

2.5.14.1 NewL

Function	static CSIPExtensionHeader* NewL (const TDesC8& aName, const TDesC8& aValue);
Description	Creates a new instance of CSIPExtensionHeader.
Preconditions	None
Parameter	const TDesC8& aName: The full or compact name of the header. const TDesC8& aValue: The value of the header.

Return value	CSIPExtensionHeader*: A new instance of CSIPExtensionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.14.2 NewLC

Function	static CSIPExtensionHeader* NewLC (const TDesC8& aName, const TDesC8& aValue);
Description	Creates a new instance of CSIPExtensionHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	const TDesC8& aName: The full or compact name of the header. const TDesC8& aValue: The value of the header.
Return value	CSIPExtensionHeader*: A new instance of CSIPExtensionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.14.3 NewL

Function	static CSIPExtensionHeader* NewL (const CSIPExtensionHeader& aExtensionHeader);
Description	Creates a deep copy of CSIPExtensionHeader.
Preconditions	None
Parameter	const CSIPExtensionHeader& aExtensionHeader: The CSIPExtensionHeader to be copied.
Return value	CSIPExtensionHeader*: A new instance of CSIPExtensionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.14.4 NewLC

Function	static CSIPExtensionHeader* NewLC (const CSIPExtensionHeader& aExtensionHeader);
Description	Creates a deep copy of CSIPExtensionHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	const CSIPExtensionHeader& aExtensionHeader: The CSIPExtensionHeader to be copied.
Return value	CSIPExtensionHeader*: A new instance of CSIPExtensionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.14.5 CloneL

Function	CSIPHeaderBase* CloneL () const;
Description	Creates a deep copy of this CSIPExtensionHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.14.6 operator==

Function	TBool operator==(const CSIPExtensionHeader& aExtensionHeader);
Description	Compares this object to another instance of CSIPExtensionHeader.
Preconditions	None
Parameter	const CSIPExtensionHeader& aExtensionHeader: A header to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.14.7 SetValueL

Function	void SetValueL (const TDesC8& aValue);
Description	Sets the header value.
Preconditions	None
Parameter	const TDesC8& aValue: The header value to be set.
Return value	void
Postconditions	None
Exceptions	The function leaves with the EErrorInExtensionHeaderValue error code if aValue contains illegal characters.

2.5.14.8 Value

Function	const TDesC8& Value () const;
Description	Gets the header value.
Preconditions	None
Parameter	None

Return value	const TDesC8&: The header value.
Postconditions	None
Exceptions	None

2.5.14.9 SetNameL

Function	void SetNameL (const TDesC8& aName);
Description	Sets the header name (used both as full and compact form name).
Preconditions	None
Parameter	const TDesC8& aName: The header name to be set.
Return value	void
Postconditions	None
Exceptions	The function leaves with the EErrorInExtensionHeaderName error code if aName is empty or contains illegal characters.

2.5.14.10 FullName

Function	const TDesC8& FullName () const;
Description	Gets the full or compact name of the header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.14.11 HasCompactName

Function	TBool HasCompactName () const;
Description	Checks if this object has a compact form name.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, the object has a compact form name.
Postconditions	None
Exceptions	None

2.5.14.12 *CompactName*

Function	const TDesC8& CompactName () const;
Description	Gets the full or compact name of the header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The name of the header.
Postconditions	None
Exceptions	None

2.5.14.13 *MoreThanOneAllowed*

Function	TBool MoreThanOneAllowed () const;
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, more than one instance of this header is allowed in a SIP message.
Postconditions	None
Exceptions	None

2.5.14.14 *IsExtensionHeader*

Function	TBool IsExtensionHeader () const;
Description	Checks if the actual class of this object is CSIPExtensionHeader.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, the actual class of this object is CSIPExtensionHeader.
Postconditions	None
Exceptions	None

2.5.14.15 *ToTextValueL*

Function	HBufC8* ToTextValueL () const;
Description	Encodes the header's value into its textual representation.
Preconditions	None
Parameter	None
Return value	HBufC8*: A textual representation of the header's value. The ownership is transferred.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.14.16 *ExternalizeSupported*

Function	TBool ExternalizeSupported () const;
Description	Checks if the header supports serialization.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, the header supports serialization.
Postconditions	None
Exceptions	None

2.5.14.17 *InternalizeValueL*

Function	static CSIPExtensionHeader* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPExtensionHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPExtensionHeader*: An instance of CSIPExtensionHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.14.18 *~CSIPExtensionHeader ()*;

Function	~CSIPExtensionHeader ();
Description	Deletes the resources of CSIPExtensionHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.15 CSIPFromHeader

This is a class for SIP “From” header manipulation. The class provides functions for setting and getting the SIP “From” header.

2.5.15.1 Name

Function	<code>static const TDesc8& Name ();</code>
Description	Gets the full name of the “From” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesc8&</code> : The full name of the header.
Postconditions	None
Exceptions	None

2.5.15.2 DecodeL

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesc8& aValue);</code>
Description	Constructs a CSIPFromHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesc8& aValue</code> : A value part of a “From” header (for example, “User <user@host>”).
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing exactly one instance of CSIPHeaderBase. Can be downcast to CSIPFromHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.15.3 NewL

Function	<code>static CSIPFromHeader* NewL (CSIPAddress* aSIPAddress);</code>
Description	Creates a new instance of CSIPFromHeader.
Preconditions	<code>aSIPAddress != 0</code> <code>aSIPAddress->Address().HostPort().HasHost() == ETrue</code>
Parameter	<code>CSIPAddress* aSIPAddress</code> : A name-address. The ownership is transferred.
Return value	<code>CSIPFromHeader*</code> : A new instance of CSIPFromHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.15.4 NewL

Function	<code>static CSIPFromHeader* NewL (CSIPAddress* aSIPAddress, const TDesC8& aTag);</code>
Description	Creates a new instance of CSIPFromHeader.
Preconditions	<code>aSIPAddress != 0</code> <code>aSIPAddress->Address().HostPort().HasHost() == ETrue</code>
Parameter	<code>CSIPAddress* aSIPAddress</code> : A name-address. The ownership is transferred. <code>const TDesC8& aTag</code> : A tag value (must be a “token” according to <i>RFC 3261</i> [1]).
Return value	<code>CSIPFromHeader*</code> : A new instance of CSIPFromHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.15.5 NewL

Function	<code>static CSIPFromHeader* NewL (const CSIPFromToHeaderBase& aFromToHeaderBase);</code>
Description	Creates a deep copy of CSIPFromToHeaderBase.
Preconditions	None
Parameter	<code>const CSIPFromToHeaderBase& aFromToHeaderBase</code> : CSIPFromToHeaderBase to be copied.
Return value	<code>CSIPFromHeader*</code> : A new instance of CSIPFromHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.15.6 NewLC

Function	<code>static CSIPFromHeader* NewLC (CSIPAddress* aSIPAddress);</code>
Description	Creates a new instance of CSIPFromHeader and puts it onto the cleanup stack.
Preconditions	<code>aSIPAddress != 0</code> <code>aSIPAddress->Address().HostPort().HasHost() == ETrue</code>
Parameter	<code>CSIPAddress* aSIPAddress</code> : A name-address. The ownership is transferred.
Return value	<code>CSIPFromHeader*</code> : A new instance of CSIPFromHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.15.7 NewLC

Function	<code>static CSIPFromHeader* NewLC (CSIPAddress* aSIPAddress, const TDesc8& aTag);</code>
Description	Creates a new instance of CSIPFromHeader CSIPFromHeader and puts it onto the cleanup stack.
Preconditions	<code>aSIPAddress != 0</code> <code>aSIPAddress->Address().HostPort().HasHost() == ETrue</code>
Parameter	<code>CSIPAddress* aSIPAddress</code> : A name-address. The ownership is transferred. <code>const TDesc8& aTag</code> : A tag value (must be a “token” according to <i>RFC 3261</i> [1]).
Return value	<code>CSIPFromHeader*</code> : A new instance of CSIPFromHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.15.8 NewLC

Function	<code>static CSIPFromHeader* NewLC (const CSIPFromToHeaderBase& aFromToHeaderBase);</code>
Description	Creates a deep copy of CSIPFromToHeaderBase and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPFromToHeaderBase& aFromToHeaderBase</code> : CSIPFromToHeaderBase to be copied.
Return value	<code>CSIPFromHeader*</code> : A new instance of CSIPFromHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.15.9 CloneL

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this CSIPFromHeader object
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.15.10 *FullName*

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “From” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.15.11 *CompactName*

Function	const TDesC8& CompactName () const;
Description	Gets the compact name of the header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The compact name of the “f” header.
Postconditions	None
Exceptions	None

2.5.15.12 *InternalizeValueL*

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPFromHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPFromHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.15.13 *~CSIPFromHeader ();*

Function	~CSIPFromHeader ();
Description	Deletes the resources of CSIPFromHeader.
Preconditions	None
Parameter	None
Return value	None

Postconditions	None
Exceptions	None

2.5.16 CSIPFromToHeaderBase

This is a base class for SIP “From” and “To” header manipulation. The class provides functions for setting and getting parameters in the SIP “From” and “To” header. This is an abstract class and cannot be instantiated.

2.5.16.1 operator==

Function	TBool operator==(const CSIPFromToHeaderBase& aSIPFromToHeaderBase) const;
Description	Compares this instance to another “From” or “To” header object.
Preconditions	None
Parameter	const CSIPFromToHeaderBase& aSIPFromToHeaderBase: A header to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.16.2 SetSIPAddressL

Function	void SetSIPAddressL (CSIPAddress* aSIPAddress);
Description	Sets the name-address.
Preconditions	aSIPAddress != 0 aSIPAddress->Address().HostPort().HasHost() == ETrue
Parameter	CSIPAddress* aSIPAddress: A name-address to set. The ownership is transferred.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.16.3 SIPAddress

Function	const CSIPAddress& SIPAddress() const;
Description	Gets the name-address as const.
Preconditions	None
Parameter	None
Return value	const CSIPAddress&: A name-address object.

Postconditions	None
Exceptions	None

2.5.16.4 SIPAddress

Function	CSIPAddress& SIPAddress() ;
Description	Gets the name-address as const.
Preconditions	None
Parameter	None
Return value	CSIPAddress&: A name-address object.
Postconditions	None
Exceptions	None

2.5.16.5 HasTag

Function	TBool HasTag () const;
Description	Checks if the tag parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present.
Postconditions	None
Exceptions	None

2.5.16.6 Tag

Function	const TDesC8& Tag () const;
Description	Gets the tag parameter.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The tag parameter if present, otherwise a zero-length descriptor.
Postconditions	None
Exceptions	None

2.5.16.7 *SetTagL*

Function	<code>void SetTagL (const TDesc8& aTag);</code>
Description	Sets the tag parameter.
Preconditions	None
Parameter	<code>const TDesc8& aTag</code> : A tag parameter to set.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.16.8 *HasCompactName*

Function	<code>TBool HasCompactName () const;</code>
Description	Checks if this object has a compact form name.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> , the object has a compact form name.
Postconditions	None
Exceptions	None

2.5.16.9 *~CSIPFromToHeaderBase ();*

Function	<code>~CSIPFromToHeaderBase ();</code>
Description	Deletes the resources of <code>CSIPFromToHeaderBase</code> .
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.17 CSIPHeaderBase

This is a base class for all the SIP headers. The class provides an interface to a generic SIP header.

2.5.17.1 CloneL

Function	<code>virtual CSIPHeaderBase* CloneL () const = 0;</code>
Description	Creates a deep copy of this CSIPHeaderBase object. The function has to be implemented in each of the subclasses.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.17.2 FullName

Function	<code>virtual const TDesc8& FullName () const = 0;</code>
Description	Gets the full name of the header. The function has to be implemented in each of the subclasses.
Preconditions	None
Parameter	None
Return value	const TDesc8&: The full name of the header, for example, "From"
Postconditions	None
Exceptions	None

2.5.17.3 HasCompactName

Function	<code>virtual TBool HasCompactName () const;</code>
Description	Checks if the actual class of this object has a compact form name. The function has to be implemented in subclasses that have a compact form name. The default implementation returns EFalse.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if the object has a compact form name, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.17.4 CompactName

Function	virtual const TDesc8& CompactName () const;
Description	Gets the full name of the header. The function has to be implemented in subclasses that have a compact form name. The default implementation returns a zero-length descriptor.
Preconditions	HasCompactName(aName) == ETrue
Parameter	None
Return value	const TDesc8&: The compact name of the header, for example “f”, if present, otherwise a zero-length descriptor.
Postconditions	None
Exceptions	None

2.5.17.5 MoreThanOneAllowed

Function	virtual TBool MoreThanOneAllowed() const;
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if more than one instance of this header is allowed in a SIP message, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.17.6 IsExtensionHeader

Function	virtual TBool IsExtensionHeader () const;
Description	Checks if the actual class of this object is CSIPExtensionHeader.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if the actual class of this object is CSIPExtensionHeader, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.17.7 ToTextL

Function	HBufC8* ToTextL () const;
Description	Encodes the header (name and value) into its textual representation.
Preconditions	None

Parameter	None
Return value	HBufC8*: A textual representation of the complete header. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.17.8 ToTextLC

Function	HBufC8* ToTextLC () const;
Description	Encodes the header (name and value) into its textual representation and puts it onto the cleanup stack.
Preconditions	None
Parameter	None
Return value	HBufC8*: A textual representation of the complete header. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.17.9 ToTextValueL

Function	virtual HBufC8* ToTextValueL () const = 0;
Description	Encodes the header's value into its textual representation.
Preconditions	None
Parameter	None
Return value	HBufC8*: A textual representation of the header's value. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.17.10 ToTextValueLC

Function	HBufC8* ToTextValueLC () const;
Description	Encodes the header's value into its textual representation and puts it onto the cleanup stack.
Preconditions	None
Parameter	None
Return value	HBufC8*: A textual representation of the header's value. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.17.11 *ExternalizeL*

Function	<code>void ExternalizeL (RWriteStream& aWriteStream, TBool aAddName=ETrue) const;</code>
Description	Writes the object to RWriteStream.
Preconditions	None
Parameter	RWriteStream& aWriteStream: A stream where the object is to be externalized. TBool aAddName=ETrue: Tells if the name of the header is written to the stream.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.17.12 *ExternalizeSupported*

Function	<code>virtual TBool ExternalizeSupported () const;</code>
Description	Checks if the header supports serialization.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if the header supports serialization, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.17.13 *PushLC*

Function	<code>static void PushLC (RPointerArray<CSIPHeaderBase>* aArray);</code>
Description	Can be used when RPointerArray<CSIPHeaderBase> needs to be pushed to the CleanupStack for ResetAndDestroy.
Preconditions	None
Parameter	RPointerArray<CSIPHeaderBase>* aArray: An array of CSIPHeaderBase pointers. The ownership of the array is transferred.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.17.14 ~CSIPHeaderBase ();

Function	~CSIPHeaderBase ();
Description	Deletes the resources of CSIPHeaderBase.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.18 CSIPHostPort

This is a class for hostport structure manipulation. The class provides functions for setting and getting host and port in hostport structure (host[":port"]) with syntax checking.

2.5.18.1 DecodeL

Function	static CSIPHostPort* DecodeL (const TDesC8& aValue);
Description	Constructs a CSIPHostPort from the textual representation.
Preconditions	None
Parameter	const TDesC8& aValue: BNF: host[":port"]
Return value	CSIPHostPort*: A new instance of CSIPHostPort.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.2 NewL

Function	static CSIPHostPort* NewL ();
Description	Creates a new CSIPHostPort instance.
Preconditions	None
Parameter	None
Return value	CSIPHostPort*: A new instance of CSIPHostPort.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.3 NewLC

Function	static CSIPHostPort* NewLC ();
Description	Creates a new CSIPHostPort instance and puts it onto the cleanup stack.
Preconditions	None

Parameter	None
Return value	CSIPHostPort*: A new instance of CSIPHostPort.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.4 NewL

Function	static CSIPHostPort* NewL (const TDesC8& aHost);
Description	Creates a new CSIPHostPort instance.
Preconditions	None
Parameter	const TDesC8& aHost: A host (host name / IPv4 / IPv6).
Return value	CSIPHostPort*: A new instance of CSIPHostPort.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.5 NewLC

Function	static CSIPHostPort* NewLC (const TDesC8& aHost);
Description	Creates a new CSIPHostPort instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	const TDesC8& aHost: A host (host name / IPv4 / IPv6).
Return value	CSIPHostPort*: A new instance of CSIPHostPort.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.6 NewL

Function	static CSIPHostPort* NewL (const TDesC8& aHost, TInt aPort);
Description	Creates a new CSIPHostPort instance.
Preconditions	None
Parameter	const TDesC8& aHost: A host (host name / IPv4 / IPv6) . TInt aPort: A port.
Return value	CSIPHostPort*: A new instance of CSIPHostPort.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.7 NewLC

Function	<code>static CSIPHostPort* NewLC (const TDesC8& aHost, TInt aPort);</code>
Description	Creates a new CSIPHostPort instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aHost</code> : A host (host name / IPv4 / IPv6). <code>TInt aPort</code> : A port.
Return value	<code>CSIPHostPort*</code> : A new instance of CSIPHostPort.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.8 NewL

Function	<code>static CSIPHostPort* NewL (const CSIPHostPort& aHostPort);</code>
Description	Creates a deep copy of the given CSIPHostPort instance.
Preconditions	None
Parameter	<code>const CSIPHostPort& aHostPort</code> : CSIPHostPort to be copied.
Return value	<code>CSIPHostPort*</code> : A new instance of CSIPHostPort.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.9 NewLC

Function	<code>static CSIPHostPort* NewLC (const CSIPHostPort& aHostPort);</code>
Description	Creates a deep copy of the given CSIPHostPort instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPHostPort& aHostPort</code> : CSIPHostPort to be copied.
Return value	<code>CSIPHostPort*</code> : A new instance of CSIPHostPort.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.10 HasHost

Function	<code>TBool HasHost () const;</code>
Description	Checks if the host part is present.
Preconditions	None
Parameter	None

Return value	TBool: ETrue if present, otherwise false.
Postconditions	None
Exceptions	None

2.5.18.11 *SetHostL*

Function	void SetHostL (const TDescC8& aHost);
Description	Sets the host.
Preconditions	None
Parameter	const TDescC8& aHost: Host (host name / IPv4 / IPv6).
Return value	void
Postconditions	None
Exceptions	The function leaves with the EErrorInHost error code if aHost is empty.

2.5.18.12 *Host*

Function	const TDescC8& Host () const;
Description	Gets the host.
Preconditions	None
Parameter	None
Return value	const TDescC8&: The host if present, otherwise a zero-length descriptor.
Postconditions	None
Exceptions	None

2.5.18.13 *HostType*

Function	TType HostType () const;
Description	Gets the type of the host.
Preconditions	HasHost() == ETrue
Parameter	None
Return value	TType: The host type (host name / IPv4 / IPv6) if present, otherwise ESIPNoHost = 0.
Postconditions	None
Exceptions	None

2.5.18.14 *HasPort*

Function	TBool HasPort () const;
Description	Checks if the port is present.
Preconditions	HasHost() == ETrue
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.18.15 *Port*

Function	TUint Port () const;
Description	Gets the port.
Preconditions	HasHost() == ETrue
Parameter	None
Return value	TUint: The port.
Postconditions	None
Exceptions	None

2.5.18.16 *SetPort*

Function	void SetPort (TUint aPort);
Description	Sets the port.
Preconditions	None
Parameter	None
Return value	void
Postconditions	None
Exceptions	None

2.5.18.17 *DeletePort*

Function	TInt DeletePort ();
Description	Deletes the port part.
Preconditions	HasPort() == ETrue
Parameter	None
Return value	TInt: KErrNotFound if port was not present, otherwise KErrNone if deletion was successful.

Postconditions	None
Exceptions	None

2.5.18.18 *operator==*

Function	TBool operator==(const CSIPHostPort& aHostPort) const;
Description	Compares this object to another instance of CSIPHostPort.
Preconditions	None
Parameter	const CSIPHostPort& aHostPort: CSIPHostPort instance to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.18.19 *ToTextLC*

Function	HBufC8* ToTextLC () const;
Description	Creates a textual representation of the CSIPHostPort instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	None
Return value	HBufC8*: A textual representation of the object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.18.20 *~CSIPHostPort ()*;

Function	~CSIPHostPort ();
Description	Deletes the resources of CSIPHostPort.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.19 CSIPParameterHeaderBase

This is a base class for SIP headers that have the structure mandatory part;parameters.

This is an abstract class and cannot be instantiated.

2.5.19.1 *HasExtensionParam*

Function	<code>TBool HasExtensionParam (const TDesc8& aName) const;</code>
Description	Checks if an extension parameter is present.
Preconditions	None
Parameter	<code>const TDesc8& aName</code> : The name of the parameter.
Return value	<code>TBool</code> : <code>ETrue</code> if present, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.19.2 *ExtensionParamValue*

Function	<code>const TDesc8& ExtensionParamValue (const TDesc8& aName) const;</code>
Description	Gets an extension parameter.
Preconditions	<code>HasExtensionParam(aName) == ETrue</code>
Parameter	<code>const TDesc8& aName</code> : The name of the parameter.
Return value	<code>const TDesc8&</code> : The parameter value if present, otherwise a zero-length descriptor.
Postconditions	None
Exceptions	None

2.5.19.3 *SetExtensionParamL*

Function	<code>void SetExtensionParamL (const TDesc8& aName, const TDesc8& aValue);</code>
Description	Sets an extension parameter.
Preconditions	<code>HasExtensionParam(aName) == EFalse</code>
Parameter	<code>const TDesc8& aName</code> : The name of the parameter. <code>const TDesc8& aValue</code> : The parameter value to set.
Return value	<code>void</code>
Postconditions	None
Exceptions	The function leaves with <code>KErrArgument</code> if <code>ParamInsertionAllowed()</code> returns <code>EFalse</code> .

2.5.19.4 *SetExtensionParamL*

Function	<code>void SetExtensionParamL (const TDesc8& aName);</code>
Description	Sets an extension parameter.
Preconditions	<code>HasExtensionParam(aName) == EFalse</code>

Parameter	const TDesC8& aName: The parameter name to set.
Return value	void
Postconditions	None
Exceptions	The function leaves with KErrArgument if ParamInsertionAllowed() returns EFalse.

2.5.19.5 DeleteExtensionParam

Function	TInt DeleteExtensionParam (const TDesC8& aName);
Description	Deletes an extension parameter.
Preconditions	HasExtensionParam(aName) == ETrue
Parameter	const TDesC8& aName: The parameter name to delete.
Return value	TInt: KErrNone if successfully deleted, KErrNotFound if the parameter was not found.
Postconditions	None
Exceptions	None

2.5.19.6 ToTextValueL

Function	HBufC8* ToTextValueL () const;
Description	Encodes the header's value into its textual representation.
Preconditions	None
Parameter	None
Return value	HBufC8*: A textual representation of the header's value. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.19.7 ~CSIPParameterHeaderBase ();

Function	~CSIPParameterHeaderBase ();
Description	Deletes the resources of CSIPParameterHeaderBase.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.20 CSIPProxyRequireHeader

This is a base class for SIP “Proxy-Require” header manipulation. The class provides functions for setting and getting parameters in the SIP “Proxy-Require” header.

2.5.20.1 Name

Function	<code>static const TDesC8& Name ();</code>
Description	Gets the full name of the “Proxy-Require” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The full name of the header.
Postconditions	None
Exceptions	None

2.5.20.2 DecodeL

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);</code>
Description	Constructs a CSIPProxyRequireHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A value part of a “Proxy-Require” header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPProxyRequireHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.20.3 CloneL

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this CSIPProxyRequireHeader object.
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.20.4 *FullName*

Function	<code>const TDesC8& FullName () const;</code>
Description	Gets the full name of the “Proxy-Require” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The full name of the header.
Postconditions	None
Exceptions	None

2.5.20.5 *NewL*

Function	<code>static CSIPProxyRequireHeader* NewL (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPProxyRequireHeader.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A tag value.
Return value	<code>CSIPProxyRequireHeader*</code> : The full name of the header.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.20.6 *NewLC*

Function	<code>static CSIPProxyRequireHeader* NewLC (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPProxyRequireHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A tag value.
Return value	<code>CSIPProxyRequireHeader*</code> : The full name of the header.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.20.7 *NewL*

Function	<code>static CSIPProxyRequireHeader* NewL (const CSIPProxyRequireHeader& aProxyRequireHeader);</code>
Description	Creates a deep copy of the given CSIPProxyRequireHeader instance.
Preconditions	None
Parameter	<code>const CSIPProxyRequireHeader& aProxyRequireHeader</code> : A CSIPProxyRequireHeader instance to be copied.

Return value	CSIPProxyRequireHeader*: A new instance of CSIPProxyRequireHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.20.8 NewLC

Function	static CSIPProxyRequireHeader* NewLC (const CSIPProxyRequireHeader& aProxyRequireHeader);
Description	Creates a deep copy of the given CSIPProxyRequireHeader instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	const CSIPProxyRequireHeader& aProxyRequireHeader: A CSIPProxyRequireHeader instance to be copied.
Return value	CSIPProxyRequireHeader*: A new instance of CSIPProxyRequireHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.20.9 operator==

Function	TBool operator==(const CSIPProxyRequireHeader& aProxyRequireHeader) const;
Description	Compares this instance to another CSIPProxyRequireHeader instance.
Preconditions	None
Parameter	const CSIPProxyRequireHeader& aProxyRequireHeader: A header to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.20.10 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPProxyRequireHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPProxyRequireHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.20.11 ~CSIPProxyRequireHeader ();

Function	~CSIPProxyRequireHeader ();
Description	Deletes the resources of CSIPProxyRequireHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.21 CSIPRequireHeader

This is a base class for SIP “Require” header manipulation. The class provides functions for setting and getting parameters in the SIP “Require” header.

2.5.21.1 Name

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Require” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.21.2 DecodeL

Function	static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);
Description	Constructs a CSIPRequireHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	const TDesC8& aValue: A value part of a “Require” header.
Return value	RPointerArray<CSIPHeaderBase>: An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPRequireHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.21.3 CloneL

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this CSIPRequireHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.21.4 FullName

Function	<code>const TDesC8& FullName () const;</code>
Description	Gets the full name of the “Require” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.21.5 NewL

Function	<code>static CSIPRequireHeader* NewL (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPRequireHeader.
Preconditions	None
Parameter	const TDesC8& aValue: A tag value.
Return value	CSIPRequireHeader*: A new instance of CSIPRequireHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.21.6 NewLC

Function	<code>static CSIPRequireHeader* NewLC (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPRequireHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	const TDesC8& aValue: A tag value.
Return value	CSIPRequireHeader*: A new instance of CSIPRequireHeader.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.21.7 NewL

Function	<code>static CSIPRequireHeader* NewL (const CSIPRequireHeader& aRequireHeader);</code>
Description	Creates a deep copy of the given CSIPRequireHeader instance.
Preconditions	None
Parameter	<code>const CSIPRequireHeader& aRequireHeader</code> : CSIPRequireHeader instance to be copied.
Return value	CSIPRequireHeader*: A new instance of CSIPRequireHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.21.8 NewLC

Function	<code>static CSIPRequireHeader* NewLC (const CSIPRequireHeader& aRequireHeader);</code>
Description	Creates a deep copy of the given CSIPRequireHeader instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPRequireHeader& aRequireHeader</code> : CSIPRequireHeader instance to be copied.
Return value	CSIPRequireHeader*: A new instance of CSIPRequireHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.21.9 operator==

Function	<code>TBool operator==(const CSIPRequireHeader& aRequireHeader) const;</code>
Description	Compares this instance to another CSIPRequireHeader instance.
Preconditions	None
Parameter	<code>const CSIPRequireHeader& aRequireHeader</code> : A header to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.21.10 *InternalizeValueL*

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPRequireHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPRequireHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.21.11 *~CSIPRequireHeader ();*

Function	~CSIPRequireHeader ();
Description	Deletes the resources of CSIPRequireHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.22 CSIPRouteHeader

This is a class for SIP “Route” header manipulation. The class provides functions for setting and getting the SIP “Route” header fields.

2.5.22.1 *Name*

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Route” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.22.2 *DecodeL*

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesc8& aValue);</code>
Description	Constructs a CSIPRouteHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesc8& aValue</code> : A value part of a "Route" header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPRouteHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.22.3 *NewL*

Function	<code>static CSIPRouteHeader* NewL (CSIPAddress* aSIPAddress);</code>
Description	Creates a new instance of CSIPRouteHeader.
Preconditions	<code>aSIPAddress != 0</code> <code>aSIPAddress->Address().HostPort().HasHost() == ETrue</code>
Parameter	<code>CSIPAddress* aSIPAddress</code> : A name-address. The ownership is transferred.
Return value	<code>CSIPRouteHeader*</code> : A new instance of CSIPRouteHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.22.4 *NewLC*

Function	<code>static CSIPRouteHeader* NewLC (CSIPAddress* aSIPAddress);</code>
Description	Creates a new instance of CSIPRouteHeader and puts it onto the cleanup stack.
Preconditions	<code>aSIPAddress != 0</code> <code>aSIPAddress->Address().HostPort().HasHost() == ETrue</code>
Parameter	<code>CSIPAddress* aSIPAddress</code> : A name-address. The ownership is transferred.
Return value	<code>CSIPRouteHeader*</code> : A new instance of CSIPRouteHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.22.5 NewL

Function	<code>static CSIPRouteHeader* NewL (const CSIPRouteHeaderBase& aRouteHeaderBase);</code>
Description	Creates a deep copy of the given CSIPRouteHeader instance.
Preconditions	None
Parameter	<code>const CSIPRouteHeaderBase& aRouteHeaderBase</code> : CSIPRouteHeader instance to be copied.
Return value	<code>CSIPRouteHeader*</code> : A new instance of CSIPRouteHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.22.6 NewLC

Function	<code>static CSIPRouteHeader* NewLC (const CSIPRouteHeaderBase& aRouteHeaderBase);</code>
Description	Creates a deep copy of the given CSIPRouteHeader instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPRouteHeaderBase& aRouteHeaderBase</code> : The header instance to be copied.
Return value	<code>CSIPRouteHeader*</code> : A new instance of CSIPRouteHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.22.7 CloneL

Function	<code>CSIPHeaderBase* CloneL () const;</code>
Description	Creates a deep copy of this CSIPRouteHeader object.
Preconditions	None
Parameter	None
Return value	<code>CSIPHeaderBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.22.8 FullName

Function	<code>const TDesC8& FullName () const;</code>
Description	Gets the full name of the "Route" header.
Preconditions	None
Parameter	None

Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.22.9 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CCSIPRouteHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPRouteHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.22.10 ExternalizeValueL

Function	void ExternalizeValueL (RWriteStream& aWriteStream) const;
Description	Writes the object to RWriteStream.
Preconditions	None
Parameter	RWriteStream& aWriteStream: A stream where the object is to be externalized.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.22.11 ~CSIPRouteHeader ();

Function	~CSIPRouteHeader ();
Description	Deletes the resources of CSIPRouteHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.23 CSIPRouteHeaderBase

This is a class for SIP “Route” and “Record-Route” header manipulation. The class provides functions for setting and getting parameters in the SIP “Route” and “Record-Route” header.

2.5.23.1 operator==

Function	TBool operator==(const CSIPRouteHeaderBase& aRouteHeaderBase) const;
Description	Compares this instance to another “Route” or “Record-Route” header object.
Preconditions	None
Parameter	const CSIPRouteHeaderBase& aRouteHeaderBase: A header to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.23.2 SetSIPAddressL

Function	void SetSIPAddressL (CSIPAddress* aSIPAddress);
Description	Sets the name-address.
Preconditions	aSIPAddress != 0 aSIPAddress->Address().HostPort().HasHost() == ETrue
Parameter	CSIPAddress* aSIPAddress: A name-address to set. The ownership is transferred.
Return value	void
Postconditions	None
Exceptions	The function leaves with EErrorInRouteOrRecordRouteHeader if aSIPAddress == 0 or aSIPAddress->URI().HostPort().HasHost() == EFalse.

2.5.23.3 SIPAddress

Function	const CSIPAddress& SIPAddress () const;
Description	Gets the name-address as const.
Preconditions	None
Parameter	None
Return value	const CSIPAddress&: The name-address.
Postconditions	None
Exceptions	None

2.5.23.4 SIPAddress

Function	CSIPAddress& SIPAddress ();
Description	Gets the name-address.
Preconditions	None
Parameter	None
Return value	CSIPAddress&: The name-address.
Postconditions	None
Exceptions	None

2.5.23.5 MoreThanOneAllowed

Function	TBool MoreThanOneAllowed () const;
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if more than one instance of this header is allowed in a SIP message, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.23.6 ~CSIPRouteHeaderBase ();

Function	~CSIPRouteHeaderBase ();
Description	Deletes the resources of CSIPRouteHeaderBase.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.24 CSIPSecurityClientHeader

This is a class for SIP “SecurityClient” header manipulation. The class provides functions for setting and getting the SIP “SecurityClient” header fields.

2.5.24.1 Name

Function	static const TDesC8& Name ();
Description	Gets the full name of the “SecurityClient” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.24.2 DecodeL

Function	static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);
Description	Constructs a CSIPSecurityClientHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	const TDesC8& aValue: A value part of a “SecurityClient” header.
Return value	RPointerArray<CSIPHeaderBase>. An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPSecurityClientHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.24.3 NewL

Function	static CSIPSecurityClientHeader* NewL (const TDesC8& aMechanismName);
Description	Creates a new instance of CSIPSecurityClientHeader.
Preconditions	None
Parameter	const TDesC8& aMechanismName: A Mechanism-Name parameter in the “SecurityClient” header.
Return value	CSIPSecurityClientHeader*: A new instance of CSIPSecurityClientHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.24.4 NewLC

Function	static CSIPSecurityClientHeader* NewLC (const TDesC8& aMechanismName);
Description	Creates a new instance of CSIPSecurityClientHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	const TDesC8& aMechanismName: A Mechanism-Name parameter in the "SecurityClient" header.
Return value	CSIPSecurityClientHeader*: A new instance of CSIPSecurityClientHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.24.5 NewL

Function	static CSIPSecurityClientHeader* NewL (const CSIPSecurityHeaderBase& aSecurityHeader);
Description	Creates a deep copy of the given CSIPSecurityHeaderBase.
Preconditions	None
Parameter	const CSIPSecurityHeaderBase& aSecurityHeader: CSIPSecurityHeaderBase to be copied.
Return value	CSIPSecurityClientHeader*: A new instance of CSIPSecurityClientHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.24.6 NewLC

Function	static CSIPSecurityClientHeader* NewLC (const CSIPSecurityHeaderBase& aSecurityHeader);
Description	Creates a deep copy of the given CSIPSecurityHeaderBase and puts it onto the cleanup stack.
Preconditions	None
Parameter	const CSIPSecurityHeaderBase& aSecurityHeader: CSIPSecurityHeaderBase to be copied.
Return value	CSIPSecurityClientHeader*: A new instance of CSIPSecurityClientHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.24.7 CloneL

Function	CSIPHeaderBase* CloneL () const;
Description	Creates a deep copy of this CSIPSecurityClientHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.24.8 FullName

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “SecurityClient” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.24.9 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPSecurityClientHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPSecurityClientHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.24.10 ~CSIPSecurityClientHeader ();

Function	~CSIPSecurityClientHeader ();
Description	Deletes the resources of CSIPSecurityClientHeader.
Preconditions	None
Parameter	None

Return value	None
Postconditions	None
Exceptions	None

2.5.25 CSIPSecurityHeaderBase

This is a base class for SIP Security headers manipulation. The class provides functions for setting and getting the SIP Security header fields: Security-Client, Security-Server, and Security-Verify.

2.5.25.1 HasPreferenceParam

Function	TBool HasPreferenceParam () const;
Description	Checks if the “q” parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.25.2 PreferenceParam

Function	TReal PreferenceParam () const;
Description	Gets the value of the “q” parameter.
Preconditions	HasPreferenceParam() == ETrue
Parameter	None
Return value	TReal: The “q” parameter value, returns zero if HasPreferenceParam() == EFalse.
Postconditions	None
Exceptions	None

2.5.25.3 SetPreferenceParamL

Function	void SetPreferenceParamL (TReal aPreferenceValue);
Description	Sets the “q” parameter value.
Preconditions	None
Parameter	TReal aPreferenceValue: A “q” parameter value to set.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.25.4 HasDigestAlgorithmParam

Function	TBool HasDigestAlgorithmParam () const;
Description	Checks if the “d-alg” parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.25.5 DigestAlgorithmParam

Function	const TDesC8& DigestAlgorithmParam () const;
Description	Gets the value of “d-alg” parameter.
Preconditions	HasDigestAlgorithmParam() == ETrue
Parameter	None
Return value	const TDesC8&: The “d-alg” parameter value.
Postconditions	None
Exceptions	None

2.5.25.6 SetDigestAlgorithmParamL

Function	void SetDigestAlgorithmParamL (const TDesC8& aDigestAlgorithmValue);
Description	Sets the “d-alg” parameter value.
Preconditions	None
Parameter	const TDesC8& aDigestAlgorithmValue: A “d-alg” parameter value to set.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.25.7 HasDigestQopParam

Function	TBool HasDigestQopParam () const;
Description	Checks if the “d-qop” parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.

Postconditions	None
Exceptions	None

2.5.25.8 *DigestQopParam*

Function	<code>const TDesC8& DigestQopParam () const;</code>
Description	Gets the value of “d-qop” parameter.
Preconditions	<code>HasDigestQopParam() == ETrue</code>
Parameter	None
Return value	<code>const TDesC8&</code> : The “d-qop” parameter value.
Postconditions	None
Exceptions	None

2.5.25.9 *SetDigestQopParamL*

Function	<code>void SetDigestQopParamL (const TDesC8& aDigestQopValue);</code>
Description	Sets the “d-qop” parameter value.
Preconditions	None
Parameter	<code>const TDesC8& aDigestQopValue</code> : A “d-qop” parameter value to set.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.25.10 *HasDigestVerifyParam*

Function	<code>TBool HasDigestVerifyParam () const;</code>
Description	Checks if the “d-ver” parameter is present.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> if present, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.25.11 *DigestVerifyParam*

Function	<code>const TDesC8& DigestVerifyParam () const;</code>
Description	Gets the value of “d-ver” parameter.
Preconditions	<code>HasDigestVerifyParam() == ETrue</code>

Parameter	None
Return value	const TDesC8&: The “d-ver” parameter value.
Postconditions	None
Exceptions	None

2.5.25.12 *SetDigestVerifyParamL*

Function	void SetDigestVerifyParamL (const TDesC8& aDigestVerifyValue);
Description	Sets the “d-ver” parameter value.
Preconditions	None
Parameter	const TDesC8& aDigestVerifyValue: A “d-ver” parameter value to set.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.25.13 *MoreThanOneAllowed*

Function	TBool MoreThanOneAllowed () const;
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if more than one instance of this header is allowed in a SIP message, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.25.14 *HasCompactName*

Function	TBool HasCompactName () const;
Description	Checks if this object has a compact form name.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if the object has a compact form name, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.25.15 *MechanismName*

Function	<code>const TDesC8& MechanismName () const;</code>
Description	Gets the Mechanism-Name parameter from the Security header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : A Mechanism-Name parameter.
Postconditions	None
Exceptions	None

2.5.25.16 *SetMechanismNameL*

Function	<code>void SetMechanismNameL (const TDesC8& aMechanismName);</code>
Description	Sets the Mechanism-Name parameter in the Security header.
Preconditions	None
Parameter	<code>const TDesC8& aMechanismName</code> : A Mechanism-Name parameter.
Return value	<code>void</code>
Postconditions	None
Exceptions	The function leaves with <code>ErrorInSecurityHeader</code> if <code>aMechanismName</code> is empty or contains invalid characters.

2.5.25.17 *~CSIPSecurityHeaderBase ()*;

Function	<code>~CSIPSecurityHeaderBase ();</code>
Description	Deletes the resources of <code>CSIPSecurityHeaderBase</code> .
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.26 *CSIPSecurityServerHeader*

This is a class for SIP “SecurityServer” header manipulation. The class provides functions for setting and getting the SIP “SecurityServer” header fields.

2.5.26.1 Name

Function	static const TDesc8& Name ();
Description	Gets the full name of the “SecurityServer” header.
Preconditions	None
Parameter	None
Return value	const TDesc8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.26.2 DecodeL

Function	static RPointerArray<CSIPHeaderBase> DecodeL (const TDesc8& aValue);
Description	Constructs a CSIPSecurityServerHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	const TDesc8& aValue: A value part of a “SecurityServer” header.
Return value	RPointerArray<CSIPHeaderBase>. An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPSecurityServerHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.26.3 NewL

Function	static CSIPSecurityServerHeader* NewL (const TDesc8& aMechanismName);
Description	Creates a new instance of CSIPSecurityServerHeader.
Preconditions	None
Parameter	const TDesc8& aMechanismName: A Mechanism-Name parameter in the “SecurityServer” header.
Return value	CSIPSecurityServerHeader*: A new instance of CSIPSecurityServerHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.26.4 NewLC

Function	static CSIPSecurityServerHeader* NewLC (const TDesC8& aMechanismName);
Description	Creates a new instance of CSIPSecurityServerHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	const TDesC8& aMechanismName: A Mechanism-Name parameter in the "SecurityServer" header.
Return value	CSIPSecurityServerHeader*: A new instance of CSIPSecurityServerHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.26.5 NewL

Function	static CSIPSecurityServerHeader* NewL (const CSIPSecurityHeaderBase& aSecurityHeader);
Description	Creates a deep copy of the given CSIPSecurityHeaderBase.
Preconditions	None
Parameter	const CSIPSecurityHeaderBase& aSecurityHeader: CSIPSecurityHeaderBase to be copied.
Return value	CSIPSecurityServerHeader*: A new instance of CSIPSecurityServerHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.26.6 NewLC

Function	static CSIPSecurityServerHeader* NewLC (const CSIPSecurityHeaderBase& aSecurityHeader);
Description	Creates a deep copy of the given CSIPSecurityHeaderBase and puts it onto the cleanup stack.
Preconditions	None
Parameter	const CSIPSecurityHeaderBase& aSecurityHeader: CSIPSecurityHeaderBase to be copied.
Return value	CSIPSecurityServerHeader*: A new instance of CSIPSecurityServerHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.26.7 CloneL

Function	CSIPHeaderBase* CloneL () const;
Description	Creates a deep copy of this CSIPSecurityServerHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.26.8 FullName

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “SecurityServer” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.26.9 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPSecurityServerHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included) .
Return value	CSIPHeaderBase*: An instance of CSIPSecurityServerHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.26.10 ~CSIPSecurityServerHeader ();

Function	~CSIPSecurityServerHeader ();
Description	Deletes the resources of CSIPSecurityServerHeader.
Preconditions	None
Parameter	None

Return value	None
Postconditions	None
Exceptions	None

2.5.27 CSIPSubscriptionStateHeader

This is a class for SIP “Subscription-State” header manipulation. The class provides functions for setting and getting parameters in the SIP “Subscription-State” header.

2.5.27.1 Name

Function	<code>static const TDesC8& Name ();</code>
Description	Gets the full name of the “Subscription-State” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The full name of the header.
Postconditions	None
Exceptions	None

2.5.27.2 DecodeL

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);</code>
Description	Constructs a CSIPSubscriptionStateHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A value part of a “Subscription-State” header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing exactly one instance of CSIPHeaderBase. Can be downcast to CSIPSubscriptionStateHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.27.3 NewL

Function	<code>static CSIPSubscriptionStateHeader* NewL (const TDesC8& aSubStateValue);</code>
Description	Creates a new instance of CSIPSubscriptionStateHeader.
Preconditions	None
Parameter	<code>const TDesC8& aSubStateValue</code> : A substate-value value.
Return value	<code>CSIPSubscriptionStateHeader*</code> : A new instance of CSIPSubscriptionStateHeader.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.27.4 NewL

Function	<code>static CSIPSubscriptionStateHeader* NewL (const CSIPSubscriptionStateHeader& aSubscriptionStateHeader);</code>
Description	Creates a deep copy of CSIPSubscriptionStateHeader.
Preconditions	None
Parameter	<code>const CSIPSubscriptionStateHeader& aSubscriptionStateHeader:</code> CSIPSubscriptionStateHeader to be copied.
Return value	CSIPSubscriptionStateHeader*: A new instance of CSIPSubscriptionStateHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.27.5 NewLC

Function	<code>static CSIPSubscriptionStateHeader* NewLC (const TDesC8& aSubStateValue);</code>
Description	Creates a new instance of CSIPSubscriptionStateHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aSubStateValue:</code> A substate-value value.
Return value	CSIPSubscriptionStateHeader*: A new instance of CSIPSubscriptionStateHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.27.6 NewLC

Function	<code>static CSIPSubscriptionStateHeader* NewLC (const CSIPSubscriptionStateHeader& aSubscriptionStateHeader);</code>
Description	Creates a deep copy of a CSIPSubscriptionStateHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPSubscriptionStateHeader& aSubscriptionStateHeader:</code> CSIPSubscriptionStateHeader to be copied.
Return value	CSIPSubscriptionStateHeader*: A new instance of CSIPSubscriptionStateHeader.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.27.7 CloneL

Function	CSIPHeaderBase* CloneL () const;
Description	Creates a deep copy of this CSIPSubscriptionStateHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.27.8 FullName

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “Subscription-State” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.27.9 SubStateValue

Function	const TDesC8& SubStateValue () const;
Description	Gets the substate-value parameter from the “Subscription-State” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The substate-value parameter.
Postconditions	None
Exceptions	None

2.5.27.10 SetSubStateValueL

Function	void SetSubStateValueL (const TDesC8& aSubStateValue);
Description	Sets the substate-value parameter in the “Subscription-State” header.
Preconditions	None

Parameter	const TDesc8& aSubStateValue: A substate-value parameter to set.
Return value	void
Postconditions	None
Exceptions	The function leaves with EErrorInSubscriptionStateHeader if aSubStateValue is empty or contains illegal characters.

2.5.27.11 HasReasonParam

Function	TBool HasReasonParam () const;
Description	Checks if the reason parameter is present in the “Subscription-State” header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.27.12 ReasonParam

Function	const TDesc8& ReasonParam () const;
Description	Gets the reason parameter from the “Subscription-State” header.
Preconditions	None
Parameter	None
Return value	const TDesc8&: The reason parameter.
Postconditions	None
Exceptions	None

2.5.27.13 SetReasonParamL

Function	void SetReasonParamL (const TDesc8& aReasonParam);
Description	Sets the reason parameter in the “Subscription-State” header.
Preconditions	None
Parameter	const TDesc8& aReasonParam: A reason parameter to set.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.27.14 *HasExpiresParameter*

Function	TBool HasExpiresParameter () const;
Description	Checks if the “expires” parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.27.15 *ExpiresParameter*

Function	TUint ExpiresParameter () const;
Description	Gets the value of the “expires” parameter.
Preconditions	HasExpiresParameter() == ETrue
Parameter	None
Return value	TUint: The “expires” parameter.
Postconditions	None
Exceptions	None

2.5.27.16 *SetExpiresParameterL*

Function	void SetExpiresParameterL (TUint aExpiresParam);
Description	Sets the “expires” parameter.
Preconditions	None
Parameter	TUint aExpiresParam: An “expires” parameter value to set.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.27.17 *HasRetryAfterParameter*

Function	TBool HasRetryAfterParameter () const;
Description	Checks if the “retry-after” parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.

Postconditions	None
Exceptions	None

2.5.27.18 *RetryAfterParameter*

Function	TUint RetryAfterParameter () const;
Description	Gets the value of the “retry-after” parameter.
Preconditions	HasRetryAfterParameter() == ETrue
Parameter	None
Return value	TUint: The “retry-after” parameter.
Postconditions	None
Exceptions	None

2.5.27.19 *SetRetryAfterParameterL*

Function	void SetRetryAfterParameterL (TUint aRetryAfterParam);
Description	Sets the “retry-after” parameter.
Preconditions	None
Parameter	TUint aRetryAfterParam: A “retry-after” parameter value to set.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.27.20 *InternalizeValueL*

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPSubscriptionStateHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPSubscriptionStateHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.27.21 ~CSIPSubscriptionStateHeader ();

Function	~CSIPSubscriptionStateHeader ();
Description	Deletes the resources of CSIPSubscriptionStateHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.28 CSIPSupportedHeader

This is a base class for SIP “Supported” header manipulation. The class provides functions for setting and getting parameters in the SIP “Supported” header.

2.5.28.1 Name

Function	static const TDesC8& Name ();
Description	Gets the full name of the “Supported” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.28.2 DecodeL

Function	static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);
Description	Constructs a CSIPSupportedHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	const TDesC8& aValue: A value part of a “Supported” header.
Return value	RPointerArray<CSIPHeaderBase>: An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPSupportedHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.28.3 NewL

Function	<code>static CSIPSupportedHeader* NewL (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPSupportedHeader.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A tag value.
Return value	<code>CSIPSupportedHeader*</code> : A new instance of CSIPSupportedHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.28.4 NewLC

Function	<code>static CSIPSupportedHeader* NewLC (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPSupportedHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A tag value.
Return value	<code>CSIPSupportedHeader*</code> : A new instance of CSIPSupportedHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.28.5 NewL

Function	<code>static CSIPSupportedHeader* NewL (const CSIPSupportedHeader& aSupportedHeader);</code>
Description	Creates a deep copy of the given CSIPSupportedHeader instance.
Preconditions	None
Parameter	<code>const CSIPSupportedHeader& aSupportedHeader</code> : A CSIPSupportedHeader to be copied.
Return value	<code>CSIPSupportedHeader*</code> : A new instance of CSIPSupportedHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.28.6 NewLC

Function	<code>static CSIPSupportedHeader* NewLC (const CSIPSupportedHeader& aSupportedHeader);</code>
Description	Creates a deep copy of the given CSIPSupportedHeader instance and puts it onto the cleanup stack.
Preconditions	None

Parameter	const CSIPSupportedHeader& aSupportedHeader: A CSIPSupportedHeader to be copied.
Return value	CSIPSupportedHeader*: A new instance of CSIPSupportedHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.28.7 operator==

Function	TBool operator==(const CSIPSupportedHeader& aSupportedHeader) const;
Description	Compares this object to another instance of CSIPSupportedHeader.
Preconditions	None
Parameter	const CSIPSupportedHeader& aSupportedHeader: A header to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.28.8 CloneL

Function	CSIPHeaderBase* CloneL () const;
Description	Creates a deep copy of this CSIPSupportedHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.28.9 FullName

Function	const TDesC8& FullName () const;
Description	Gets the full name of the "Supported" header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.28.10 *HasCompactName*

Function	TBool HasCompactName () const;
Description	Checks if this object has a compact form name.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, the object has a compact form name.
Postconditions	None
Exceptions	None

2.5.28.11 *CompactName*

Function	const TDesC8& CompactName () const;
Description	Gets the compact name of the “k” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The compact name of the “k” header.
Postconditions	None
Exceptions	None

2.5.28.12 *InternalizeValueL*

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPSupportedHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPSupportedHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.28.13 *HasValue*

Function	TBool HasValue () const;
Description	
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.

Postconditions	None
Exceptions	None

2.5.28.14 ~CSIPSupportedHeader ();

Function	~CSIPSupportedHeader ();
Description	Deletes the resources of CSIPSupportedHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.29 CSIPToHeader

This is a class for SIP “To” header manipulation. The class provides functions for setting and getting parameters in the SIP “To” header.

2.5.29.1 Name

Function	static const TDesc8& Name ();
Description	Gets the full name of the “To” header.
Preconditions	None
Parameter	None
Return value	const TDesc8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.29.2 DecodeL

Function	static RPointerArray<CSIPHeaderBase> DecodeL (const TDesc8& aValue);
Description	Constructs a CSIPToHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	const TDesc8& aValue: A value part of a “To” header (for example, "User <user@host>").
Return value	RPointerArray<CSIPHeaderBase>: An array containing exactly one instance of CSIPHeaderBase. Can be downcast to CSIPToHeader subtype if needed.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.29.3 NewL

Function	<code>static CSIPToHeader* NewL (CSIPAddress* aSIPAddress);</code>
Description	Creates a new instance of CSIPToHeader.
Preconditions	<code>aSIPAddress != 0</code> <code>aSIPAddress->Address().HostPort().HasHost() == ETrue</code>
Parameter	<code>CSIPAddress* aSIPAddress</code> : A name-address. The ownership is transferred.
Return value	<code>CSIPToHeader*</code> : A new instance of CSIPToHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.29.4 NewL

Function	<code>static CSIPToHeader* NewL (CSIPAddress* aSIPAddress, const TDesc8& aTag);</code>
Description	Creates a new instance of CSIPToHeader.
Preconditions	<code>aSIPAddress != 0</code> <code>aSIPAddress->Address().HostPort().HasHost() == ETrue</code>
Parameter	<code>CSIPAddress* aSIPAddress</code> : A name-address. The ownership is transferred. <code>const TDesc8& aTag</code> : A tag value (must be a “token” according to <i>RFC 3261</i> [1]).
Return value	<code>CSIPToHeader*</code> : A new instance of CSIPToHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.29.5 NewL

Function	<code>static CSIPToHeader* NewL (const CSIPFromToHeaderBase& aFromToHeaderBase);</code>
Description	Creates a deep copy of CSIPFromToHeaderBase.
Preconditions	None
Parameter	<code>const CSIPFromToHeaderBase& aFromToHeaderBase</code> : CSIPFromToHeaderBase to be copied.
Return value	<code>CSIPToHeader*</code> : A new instance of CSIPFromHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.29.6 NewLC

Function	<code>static CSIPToHeader* NewLC (CSIPAddress* aSIPAddress);</code>
Description	Creates a new instance of CSIPToHeader and puts it onto the cleanup stack.
Preconditions	<code>aSIPAddress != 0</code> <code>aSIPAddress->Address().HostPort().HasHost() == ETrue</code>
Parameter	<code>CSIPAddress* aSIPAddress</code> : A name-address. The ownership is transferred.
Return value	<code>CSIPToHeader*</code> : A new instance of CSIPToHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.29.7 NewLC

Function	<code>static CSIPToHeader* NewLC (CSIPAddress* aSIPAddress, const TDesC8& aTag);</code>
Description	Creates a new instance of CSIPToHeader and puts it onto the cleanup stack.
Preconditions	<code>aSIPAddress != 0</code> <code>aSIPAddress->Address().HostPort().HasHost() == ETrue</code>
Parameter	<code>CSIPAddress* aSIPAddress</code> : A name-address. The ownership is transferred. <code>const TDesC8& aTag</code> : A tag value (must be a “token” according to <i>RFC 3261</i> [1]).
Return value	<code>CSIPToHeader*</code> : A new instance of CSIPToHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.29.8 NewLC

Function	<code>static CSIPToHeader* NewLC (const CSIPFromToHeaderBase& aFromToHeaderBase);</code>
Description	Creates a deep copy of a CSIPFromToHeaderBase and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPFromToHeaderBase& aFromToHeaderBase</code> : CSIPFromToHeaderBase to be copied.
Return value	<code>CSIPToHeader*</code> : A new instance of CSIPToHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.29.9 *CloneL*

Function	CSIPHeaderBase* CloneL () const;
Description	Creates a deep copy of this CSIPToHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.29.10 *FullName*

Function	const TDesC8& FullName () const;
Description	Gets the full name of the “To” header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.29.11 *CompactName*

Function	const TDesC8& CompactName () const;
Description	Gets the compact name of the header.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The compact name of the “t” header.
Postconditions	None
Exceptions	None

2.5.29.12 *InternalizeValueL*

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPToHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).
Return value	CSIPHeaderBase*: An instance of CSIPToHeader.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.29.13 ~CSIPToHeader ();

Function	~CSIPToHeader ();
Description	Deletes the resources of CSIPToHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.30 CSIPTokenHeaderBase

This is a base class for SIP headers having a single token value. The class provides an interface to set and get the token value.

2.5.30.1 SetValueL

Function	void SetValueL (const TDesC8& aValue);
Description	Sets the header value as a token.
Preconditions	None
Parameter	const TDesC8& aValue: A value to set.
Return value	void
Postconditions	None
Exceptions	The function leaves with EErrorInTokenBaseHeader if aValue is empty or contains invalid characters.

2.5.30.2 Value

Function	const TDesC8& Value () const;
Description	Gets the header value as a token.
Preconditions	None
Parameter	None
Return value	const TDesC8&: The header value.
Postconditions	None
Exceptions	None

2.5.30.3 *ToTextValueL*

Function	<code>virtual HBufC8* ToTextValueL () const = 0;</code>
Description	Encodes the header's value into its textual representation.
Preconditions	None
Parameter	None
Return value	HBufC8*: A textual representation of the header's value. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.30.4 *MoreThanOneAllowed*

Function	<code>TBool MoreThanOneAllowed () const;</code>
Description	Checks if a SIP message may have more than one instance of this header.
Preconditions	None
Parameter	None
Return value	TBool: ETrue, more than one instance of this header is allowed in a SIP message.
Postconditions	None
Exceptions	None

2.5.30.5 *~CSIPTokenHeaderBase ()*;

Function	<code>~CSIPTokenHeaderBase ();</code>
Description	Deletes the resources of CSIPTokenHeaderBase.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.31 CSIPUnsignedIntHeaderBase

This is a base class for SIP headers having a single unsigned integer value. The class provides an interface to set and get the unsigned integer value.

2.5.31.1 SetValue

Function	void SetValue (TUint aValue);
Description	Sets the header value as an unsigned integer.
Preconditions	None
Parameter	TUint aValue: A value to set.
Return value	void
Postconditions	None
Exceptions	None

2.5.31.2 Value

Function	TUint Value () const;
Description	Gets the header value as an unsigned integer.
Preconditions	None
Parameter	None
Return value	TUint: The header value.
Postconditions	None
Exceptions	None

2.5.31.3 ToTextValueL

Function	HBufC8* ToTextValueL () const;
Description	Encodes the header's value into its textual representation.
Preconditions	None
Parameter	None
Return value	HBufC8*: A textual representation of the header's value. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.31.4 ~CSIPUnsignedIntHeaderBase ();

Function	~CSIPUnsignedIntHeaderBase ();
Description	Deletes the resources of CSIPUnsignedIntHeaderBase.
Preconditions	None
Parameter	None
Return value	None

Postconditions	None
Exceptions	None

2.5.32 CSIPUnsupportedHeader

This is a base class for SIP “Unsupported” header manipulation. The class provides functions for setting and getting parameters in the SIP “Unsupported” header.

2.5.32.1 Name

Function	<code>static const TDesC8& Name ();</code>
Description	Gets the full name of the “Unsupported” header.
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The full name of the header.
Postconditions	None
Exceptions	None

2.5.32.2 DecodeL

Function	<code>static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);</code>
Description	Constructs a CSIPUnsupportedHeader from the textual representation of the header's value part.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A value part of a “Unsupported” header.
Return value	<code>RPointerArray<CSIPHeaderBase></code> : An array containing one to many instances of CSIPHeaderBase. Can be downcast to CSIPUnsupportedHeader subtype if needed.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.32.3 NewL

Function	<code>static CSIPUnsupportedHeader* NewL (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPUnsupportedHeader.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A tag value.
Return value	<code>CSIPUnsupportedHeader*</code> : A new instance of CSIPUnsupportedHeader.

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.32.4 NewLC

Function	<code>static CSIPUnsupportedHeader* NewLC (const TDesC8& aValue);</code>
Description	Creates a new instance of CSIPUnsupportedHeader and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : A tag value.
Return value	CSIPUnsupportedHeader*: A new instance of CSIPUnsupportedHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.32.5 NewL

Function	<code>static CSIPUnsupportedHeader* NewL (const CSIPUnsupportedHeader& aUnsupportedHeader);</code>
Description	Creates a deep copy of the given CSIPUnsupportedHeader instance.
Preconditions	None
Parameter	<code>const CSIPUnsupportedHeader& aUnsupportedHeader</code> : A CSIPUnsupportedHeader to be copied.
Return value	CSIPUnsupportedHeader*: A new instance of CSIPUnsupportedHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.32.6 NewLC

Function	<code>static CSIPUnsupportedHeader* NewLC (const CSIPUnsupportedHeader& aUnsupportedHeader);</code>
Description	Creates a deep copy of the given CSIPUnsupportedHeader instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	<code>const CSIPUnsupportedHeader& aUnsupportedHeader</code> : A CSIPUnsupportedHeader to be copied.
Return value	CSIPUnsupportedHeader*: A new instance of CSIPUnsupportedHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.32.7 operator==

Function	TBool operator==(const CSIPUnsupportedHeader& aUnsupportedHeader) const;
Description	Compares this object to another instance of CSIPUnsupportedHeader.
Preconditions	None
Parameter	const CSIPUnsupportedHeader& aUnsupportedHeader: A header to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.32.8 CloneL

Function	CSIPHeaderBase* CloneL () const;
Description	Creates a deep copy of this CSIPUnsupportedHeader object.
Preconditions	None
Parameter	None
Return value	CSIPHeaderBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.32.9 FullName

Function	const TDesc8& FullName () const;
Description	Gets the full name of the “Unsupported” header.
Preconditions	None
Parameter	None
Return value	const TDesc8&: The full name of the header.
Postconditions	None
Exceptions	None

2.5.32.10 InternalizeValueL

Function	static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPUnsupportedHeader from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the value of the externalized header object (header name not included).

Return value	CSIPHeaderBase*: An instance of CSIPUnsupportedHeader.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.32.11 ~CSIPUnsupportedHeader ();

Function	~CSIPUnsupportedHeader ();
Description	Deletes the resources of CSIPUnsupportedHeader.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.33 CSIPURI

This is a class for SIP-URI or SIPS-URI structure manipulation. The class provides functions for setting and getting fields and parameters in the SIP-URI or SIPS-URI structure.

2.5.33.1 DecodeL

Function	static CSIPURI* DecodeL (const TDesC8& aValue);
Description	Constructs a CSIPURI from the textual representation.
Preconditions	None
Parameter	const TDesC8& aValue: A SIP-URI or SIPS-URI as text.
Return value	CSIPURI*: A new instance of CSIPURI.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.2 NewL

Function	static CSIPURI* NewL (CSIPHostPort* aHostPort, TBool aIsSIPsUri=EFalse);
Description	Creates a new CSIPURI instance.
Preconditions	aHostPort != 0
Parameter	CSIPHostPort* aHostPort: the ownership is transferred. TBool aIsSIPsUri=EFalse: If EFalse scheme is set to "sip:" If ETrue scheme is set to "sips:"
Return value	CSIPURI*: A new instance of CSIPURI.

Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with the EErrorInHostPort error code if the precondition is not met.

2.5.33.3 NewLC

Function	static CSIPURI* NewLC (CSIPHostPort* aHostPort, TBool aIsSIPUri=EFalse);
Description	Creates a new CSIPURI instance and puts it onto the cleanup stack.
Preconditions	aHostPort != 0
Parameter	CSIPHostPort* aHostPort: The ownership is transferred, @see CSIPHostPort. TBool aIsSIPUri=EFalse: If EFalse scheme is set to "sip:" If ETrue scheme is set to "sips:"
Return value	CSIPURI*: A new instance of CSIPURI.
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with the EErrorInHostPort error code if the precondition is not met.

2.5.33.4 NewL

Function	static CSIPURI* NewL (const TDesc8& aUser, CSIPHostPort* aHostPort, TBool aIsSIPUri=EFalse);
Description	Creates a new CSIPURI instance.
Preconditions	aHostPort != 0
Parameter	const TDesc8& aUser: The user. CSIPHostPort* aHostPort: The ownership is transferred, @see CSIPHostPort. TBool aIsSIPUri=EFalse: If EFalse scheme is set to "sip:" If ETrue scheme is set to "sips:"
Return value	CSIPURI*: A new instance of CSIPURI.
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with the EErrorInHostPort error code if the precondition is not met.

2.5.33.5 NewLC

Function	static CSIPURI* NewLC (const TDesc8& aUser, CSIPHostPort* aHostPort, TBool aIsSIPUri=EFalse);
Description	Creates a new CSIPURI instance and puts it onto the cleanup stack.
Preconditions	aHostPort != 0
Parameter	const TDesc8& aUser: The user. CSIPHostPort* aHostPort: The ownership is transferred, @see

	CSIPHostPort. TBool aIsSIPURI=EFALSE: If EFALSE scheme is set to "sip:" If ETRUE scheme is set to "sips:"
Return value	CSIPURI*: A new instance of CSIPURI.
Postconditions	None
Exceptions	In case of an error, the function leaves. The function leaves with the EErrorInHostPort error code if the precondition is not met.

2.5.33.6 NewL

Function	static CSIPURI* NewL (const CSIPURI& aSIPURI);
Description	Creates a deep copy of the given CSIPURI instance.
Preconditions	None
Parameter	const CSIPURI& aSIPURI: CSIPURI to be copied.
Return value	CSIPURI*: A new instance of CSIPURI.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.7 NewLC

Function	static CSIPURI* NewLC (const CSIPURI& aSIPURI);
Description	Creates a deep copy of the given CSIPURI instance and puts it onto the cleanup stack.
Preconditions	None
Parameter	const CSIPURI& aSIPURI: CSIPURI to be copied.
Return value	CSIPURI*: A new instance of CSIPURI.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.8 IsSIPSURI

Function	TBool IsSIPSURI () const;
Description	Checks the scheme of this CSIPURI.
Preconditions	None
Parameter	None
Return value	TBool: ETRUE if the scheme is "sips:", EFALSE if the scheme is "sip:".
Postconditions	None
Exceptions	None

2.5.33.9 *SetHostPortL*

Function	<code>void SetHostPortL (CSIPHostPort* aHostPort);</code>
Description	Sets the hostport.
Preconditions	<code>aHostPort != 0</code>
Parameter	<code>CSIPHostPort* aHostPort</code> : The hostport. The ownership is transferred.
Return value	<code>void</code>
Postconditions	None
Exceptions	The function leaves with <code>EErrorInHost</code> if <code>aHostPort == 0</code> .

2.5.33.10 *HostPort*

Function	<code>const CSIPHostPort& HostPort () const;</code>
Description	Gets the hostport as const.
Preconditions	None
Parameter	None
Return value	<code>const CSIPHostPort&</code> : The hostport.
Postconditions	None
Exceptions	None

2.5.33.11 *HostPort*

Function	<code>CSIPHostPort& HostPort () ;</code>
Description	Gets the hostport.
Preconditions	None
Parameter	None
Return value	<code>CSIPHostPort&</code> : The hostport.
Postconditions	None
Exceptions	None

2.5.33.12 *HasUser*

Function	<code>TBool HasUser () const;</code>
Description	Checks if the user part of the userinfo structure is present.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> if present, otherwise <code>EFalse</code> .

Postconditions	None
Exceptions	None

2.5.33.13 *User*

Function	<code>const TDesc8& User () const;</code>
Description	Gets the user part of the userinfo structure.
Preconditions	<code>HasUser() == ETrue</code>
Parameter	None
Return value	<code>const TDesc8&</code> : The user.
Postconditions	None
Exceptions	None

2.5.33.14 *SetUserL*

Function	<code>void SetUserL (const TDesc8& aUser);</code>
Description	Sets the user part of the userinfo structure.
Preconditions	None
Parameter	<code>const TDesc8& aUser</code> : The user to set.
Return value	<code>void</code>
Postconditions	None
Exceptions	The function leaves with <code>EErrorInSIPURIUser</code> if <code>aUser</code> is empty or contains illegal characters.

2.5.33.15 *DeleteUserInfo*

Function	<code>void DeleteUserInfo ();</code>
Description	Deletes the userinfo (user+password).
Preconditions	None
Parameter	None
Return value	<code>void</code>
Postconditions	None
Exceptions	None

2.5.33.16 *HasPassword*

Function	TBool HasPassword () const;
Description	Checks if the password part of the userinfo structure is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.33.17 *Password*

Function	const TDesC8& Password () const;
Description	Gets the password part of the userinfo structure.
Preconditions	HasPassword() == ETrue
Parameter	None
Return value	const TDesC8&: The password.
Postconditions	None
Exceptions	None

2.5.33.18 *SetPasswordL*

Function	void SetPasswordL (const TDesC8& aPassword);
Description	Sets the password part of the userinfo structure.
Preconditions	HasUser() == ETrue
Parameter	const TDesC8& aPassword: The password to set.
Return value	void
Postconditions	None
Exceptions	The function leaves with EErrorInSIPURIPassword if aPassword is empty or contains illegal characters. The function leaves with KErrArgument if iHasUser == EFalse.

2.5.33.19 *DeletePassword*

Function	void DeletePassword ();
Description	Deletes the password part of the userinfo structure.
Preconditions	None
Parameter	None
Return value	void

Postconditions	None
Exceptions	None

2.5.33.20 *HasParams*

Function	TBool HasParams () const;
Description	Checks if any parameters are present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.33.21 *HasTransportParam*

Function	TBool HasTransportParam () const;
Description	Checks if the “transport” parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.33.22 *TransportParam*

Function	const TDesc8& TransportParam () const;
Description	Gets the value of the “transport” parameter.
Preconditions	HasTransportParam() == ETrue
Parameter	None
Return value	const TDesc8&: The value of the “transport” parameter if present, otherwise a zero-length descriptor.
Postconditions	None
Exceptions	None

2.5.33.23 *SetTransportParamL*

Function	<code>void SetTransportParamL (const TDesC8& aValue);</code>
Description	Sets the value of the “transport” parameter.
Preconditions	None
Parameter	<code>const TDesC8& aValue</code> : The value of the “transport” parameter.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.24 *DeleteTransportParam*

Function	<code>TInt DeleteTransportParam ();</code>
Description	Deletes the “transport” parameter.
Preconditions	<code>HasTransportParam() == ETrue</code>
Parameter	None
Return value	<code>TInt</code> : <code>KErrNotFound</code> if the “transport” parameter was not present, otherwise <code>KErrNone</code> .
Postconditions	None
Exceptions	None

2.5.33.25 *HasUserParam*

Function	<code>TBool HasUserParam () const;</code>
Description	Checks if the “user” parameter is present.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> if present, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.33.26 *UserParam*

Function	<code>const TDesC8& UserParam () const;</code>
Description	Gets the value of the “user” parameter.
Preconditions	<code>HasUserParam() == ETrue</code>
Parameter	None
Return value	<code>const TDesC8&</code> : The value of the “user” parameter.

Postconditions	None
Exceptions	None

2.5.33.27 *SetUserParamL*

Function	<code>void SetUserParamL (const TDesc8& aValue);</code>
Description	Sets the value of the “user” parameter.
Preconditions	None
Parameter	<code>const TDesc8& aValue</code> : The value of the “user” parameter.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.28 *DeleteUserParam*

Function	<code>TInt DeleteUserParam ();</code>
Descriptions	Deletes the “user” parameter.
Preconditions	<code>HasUserParam() == ETrue</code>
Parameter	None
Return value	<code>TInt</code> : <code>KErrNotFound</code> if the “user” parameter was not present, otherwise <code>KErrNone</code> .
Postconditions	None
Exceptions	None

2.5.33.29 *HasMethodParam*

Function	<code>TBool HasMethodParam () const;</code>
Descriptions	Checks if the “method” parameter is present.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> if present, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.33.30 *MethodParam*

Function	<code>const TDesc8& MethodParam () const;</code>
Description	Gets the value of the “method” parameter.
Preconditions	<code>HasMethodParam() == ETrue</code>
Parameter	None
Return value	<code>const TDesc8&</code> : The value of the “method” parameter.
Postconditions	None
Exceptions	None

2.5.33.31 *SetMethodParamL*

Function	<code>void SetMethodParamL (const TDesc8& aValue);</code>
Description	Sets the value of the “method” parameter.
Preconditions	None
Parameter	<code>const TDesc8& aValue</code> : The value of the “method” parameter.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.32 *DeleteMethodParam*

Function	<code>TInt DeleteMethodParam ();</code>
Description	Deletes the “method” parameter.
Preconditions	<code>HasMethodParam() == ETrue</code>
Parameter	None
Return value	<code>TInt</code> : <code>KErrNotFound</code> if the “method” parameter was not present, otherwise <code>KErrNone</code> .
Postconditions	None
Exceptions	None

2.5.33.33 *HasTtlParam*

Function	<code>TBool HasTtlParam () const;</code>
Description	Checks if the “ttl” parameter is present.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> if present, otherwise <code>EFalse</code> .

Postconditions	None
Exceptions	None

2.5.33.34 *TtlParam*

Function	TUint TtlParam () const;
Description	Gets the value of the “ttl” parameter.
Preconditions	HasTtlParam() == ETrue
Parameter	None
Return value	TUint: The value of the “ttl” parameter.
Postconditions	None
Exceptions	None

2.5.33.35 *SetTtlParamL*

Function	void SetTtlParamL (TUint aValue);
Description	Sets the value of the “ttl” parameter.
Preconditions	None
Parameter	TUint aValue: The value of the “ttl” parameter.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.36 *DeleteTtlParam*

Function	TInt DeleteTtlParam ();
Description	Deletes the “ttl” parameter.
Preconditions	HasTtlParam() == ETrue
Parameter	None
Return value	TInt: KErrNotFound if the “ttl” parameter was not present, otherwise KErrNone.
Postconditions	None
Exceptions	None

2.5.33.37 *HasMaddrParam*

Function	TBool HasMaddrParam () const;
Description	Checks if the “maddr” parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.33.38 *MaddrParam*

Function	const TDesc8& MaddrParam () const;
Description	Gets the value of the “maddr” parameter.
Preconditions	HasMaddrParam() == ETrue
Parameter	None
Return value	const TDesc8&: The value of the “maddr” parameter.
Postconditions	None
Exceptions	None

2.5.33.39 *SetMaddrParamL*

Function	void SetMaddrParamL (const TDesc8& aValue);
Description	Sets the value of the “maddr” parameter.
Preconditions	None
Parameter	const TDesc8& aValue: The value of the “maddr” parameter.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.40 *DeleteMaddrParam*

Function	TInt DeleteMaddrParam ();
Description	Deletes the “maddr” parameter.
Preconditions	HasMaddrParam() == ETrue
Parameter	void
Return value	TInt: KErrNotFound if the “maddr” parameter was not present, otherwise KErrNone.

Postconditions	None
Exceptions	None

2.5.33.41 *HasLrParam*

Function	TBool HasLrParam () const;
Description	Checks if the “lr” parameter is present.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.33.42 *SetLrParamL*

Function	void SetLrParamL ();
Description	Sets the “lr” parameter (note that “lr” does not have a value).
Preconditions	None
Parameter	None
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.43 *DeleteLrParam*

Function	TInt DeleteLrParam ();
Description	Deletes the “lr” parameter.
Preconditions	HasLrParam() == ETrue
Parameter	None
Return value	TInt: KErrNotFound if the “lr” parameter was not present, otherwise KErrNone.
Postconditions	None
Exceptions	None

2.5.33.44 *HasOtherURIParam*

Function	TBool HasOtherURIParam (const TDesc8& aName) const;
Description	Checks if a named other URI parameter is present.
Preconditions	None
Parameter	const TDesc8& aName: The name of the other URI parameter.
Return value	TBool: ETrue if present, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.33.45 *OtherURIParamValue*

Function	const TDesc8& OtherURIParamValue (const TDesc8& aName) const;
Description	Gets the value of another URI parameter.
Preconditions	HasOtherURIParam(aName) == ETrue
Parameter	const TDesc8& aName: The name of the other URI parameter.
Return value	const TDesc8&: The value of another URI parameter.
Postconditions	None
Exceptions	None

2.5.33.46 *SetOtherURIParamL*

Function	void SetOtherURIParamL (const TDesc8& aName);
Description	Sets another new URI parameter that does not have a value.
Preconditions	None
Parameter	const TDesc8& aName: The name of the other URI parameter.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.47 *SetOtherURIParamL*

Function	void SetOtherURIParamL (const TDesc8& aName, const TDesc8& aValue);
Description	Sets another new URI parameter with a name and a value.
Preconditions	None
Parameter	const TDesc8& aName: The name of the other URI parameter. const TDesc8& aValue: The value of the other URI parameter.
Return value	void

Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.48 *DeleteOtherURIParam*

Function	<code>TInt DeleteOtherURIParam (const TDesc8& aName);</code>
Description	Deletes another URI parameter.
Preconditions	<code>HasOtherURIParam(aName) == ETrue</code>
Parameter	<code>const TDesc8& aName</code> : The name of the other URI parameter.
Return value	<code>TInt</code> : <code>KErrNotFound</code> if the other URI parameter was not present, otherwise <code>KerrNone</code> . If the deletion was successful.
Postconditions	None
Exceptions	None

2.5.33.49 *HasURIHeaders*

Function	<code>TBool HasURIHeaders (const TDesc8& aName) const;</code>
Description	Checks if named URI headers are present.
Preconditions	None
Parameter	<code>const TDesc8& aName</code> : The name of the URI headers.
Return value	<code>TBool</code> : <code>ETrue</code> if present, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.33.50 *URIHeaderValuesL*

Function	<code>CPtrC8Array* URIHeaderValuesL (const TDesc8& aName) const;</code>
Description	Returns an array of URI headers with the given name. The header values are copies of the actual header values in the SIP-URI. Thus, the actual values cannot be modified through the given array. Note that removing a header value from the array does NOT remove it from the SIP URI.
Preconditions	None
Parameter	<code>const TDesc8& aName</code> : The name of the URI headers.
Return value	<code>CPtrC8Array*</code> : 0 if no URI headers with the given name were found. Otherwise array of URI header values. The ownership of the array is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.51 *AddURIHeaderL*

Function	<code>void AddURIHeaderL (const TDesC8& aName, const TDesC8& aValue);</code>
Description	Adds a new URI header.
Preconditions	None
Parameter	<code>const TDesC8& aName</code> : The name of the URI headers. <code>const TDesC8& aValue</code> : The value of the URI headers.
Return value	<code>void</code>
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.52 *operator==*

Function	<code>TBool operator==(const CURIBase& aURI) const;</code>
Description	Compares this object to another instance of CURIBase.
Preconditions	None
Parameter	<code>const CURIBase& aURI</code> : A CURIBase to compare to.
Return value	<code>TBool</code> : <code>ETrue</code> if the objects are equal, otherwise <code>EFalse</code> .
Postconditions	None
Exceptions	None

2.5.33.53 *CloneL*

Function	<code>CURIBase* CloneL () const;</code>
Description	Creates a deep copy of this CSIPURI object.
Preconditions	None
Parameter	None
Return value	<code>CURIBase*</code> : The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.54 *IsSIPURI*

Function	<code>TBool IsSIPURI () const;</code>
Description	Checks if the actual class of this object is CSIPURI.
Preconditions	None
Parameter	None
Return value	<code>TBool</code> : <code>ETrue</code> , the object is of class CSIPURI.

Postconditions	None
Exceptions	None

2.5.33.55 *ToTextL*

Function	HBufC8* ToTextL () const;
Description	Encodes the object into a textual representation.
Preconditions	None
Parameter	None
Return value	HBufC8*: a textual representation of the object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.56 *InternalizeL*

Function	static CSIPURI* InternalizeL (RReadStream& aReadStream);
Description	Constructs an instance of CSIPURI from RReadStream.
Preconditions	None
Parameter	RReadStream& aReadStream: A stream containing the externalized object.
Return value	CSIPURI*: An instance of CSIPURI.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.57 *ExternalizeL*

Function	void ExternalizeL (RWriteStream& aWriteStream) const;
Description	Externalizes the object to stream.
Preconditions	None
Parameter	RWriteStream& aWriteStream: A stream where the object is to be externalized.
Return value	void
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.33.58 ~CSIPURI ();

Function	~CSIPURI ();
Description	Deletes the resources of CSIPURI.
Preconditions	None
Parameter	None
Return value	None
Postconditions	None
Exceptions	None

2.5.34 CURIBase

This is a base class for URI structure manipulation. The class provides an interface to a generic URI structure.

2.5.34.1 CloneL

Function	virtual CURIBase* CloneL () const = 0;
Description	Creates a deep copy of this CURIBase object. The function has to be implemented in each of the subclasses.
Preconditions	None
Parameter	None
Return value	CURIBase*: The deep-copied object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.34.2 IsSIPURI

Function	virtual TBool IsSIPURI () const = 0;
Description	Checks if the actual class of this object is CSIPURI. The function has to be implemented in each of the subclasses.
Preconditions	None
Parameter	None
Return value	TBool: ETrue if the object is of class CSIPURI.
Postconditions	None
Exceptions	None

2.5.34.3 ToTextL

Function	<code>virtual HBufC8* ToTextL () const = 0;</code>
Description	Encodes the object into a textual representation. The function has to be implemented in each of the subclasses.
Preconditions	None
Parameter	None
Return value	HBufC8*: a textual representation of the object. The ownership is transferred.
Postconditions	None
Exceptions	In case of an error, the function leaves.

2.5.34.4 operator==

Function	<code>virtual TBool operator==(const CURIBase& aURI) const = 0;</code>
Description	Compares this object to another object also having CURIBase base class. The function has to be implemented in each of the subclasses.
Preconditions	None
Parameter	<code>const CURIBase& aURI</code> : a URI object to compare to.
Return value	TBool: ETrue if the objects are equal, otherwise EFalse.
Postconditions	None
Exceptions	None

2.5.34.5 Scheme

Function	<code>const TDesC8& Scheme () const;</code>
Description	Gets the scheme part of the URI (for example, "sip", "mailto", ...).
Preconditions	None
Parameter	None
Return value	<code>const TDesC8&</code> : The scheme part.
Postconditions	None
Exceptions	None

2.5.34.6 ~CURIBase ();

Function	<code>~CURIBase ();</code>
Description	Deletes the resources of CURIBase.
Preconditions	None
Parameter	None

Return value	None
Postconditions	None
Exceptions	None

2.6 Code Architecture

<code>sipacceptencodingheader.h</code>	<code>class CSIPAcceptEncodingHeader</code>
<code>sipacceptheader.h</code>	<code>class CSIPAcceptHeader</code>
<code>sipacceptlanguageheader.h</code>	<code>class CSIPAcceptLanguageHeader</code>
<code>sipaddress.h</code>	<code>class CSIPAddress</code>
<code>sipalloweventsheader.h</code>	<code>class CSIPAllowEventsHeader</code>
<code>sipallowheader.h</code>	<code>class CSIPAllowHeader</code>
<code>sipcontactheader.h</code>	<code>class CSIPContactHeader</code>
<code>sipcontentdispositionheader.h</code>	<code>class CSIPContentDispositionHeader</code>
<code>sipcontentencodingheader.h</code>	<code>class CSIPContentEncodingHeader</code>
<code>sipcontenttypeheader.h</code>	<code>class CSIPContentTypeHeader</code>
<code>sipcseqheader.h</code>	<code>class CSIPCSeqHeader</code>
<code>sipeventheader.h</code>	<code>class CSIPEventHeader</code>
<code>SipExpiresHeader.h</code>	<code>class CSIPExpiresHeader</code>
<code>sipextensionheader.h</code>	<code>class CSIPExtensionHeader</code>
<code>sipfromheader.h</code>	<code>class CSIPFromHeader</code>
<code>sipfromtoheaderbase</code>	<code>class CSIPFromToHeaderBase</code>
<code>sipheaderbase.h</code>	<code>class CSIPHeaderBase</code>
<code>siphostport.h</code>	<code>class CSIPHostPort</code>
<code>SipParameterHeaderBase.h</code>	<code>class CSIPParameterHeaderBase</code>
<code>sipproxyrequireheader.h</code>	<code>class CSIPProxyRequireHeader</code>
<code>siprequireheader.h</code>	<code>class CSIPRequireHeader</code>
<code>SipRouteHeader.h</code>	<code>class CSIPRouteHeader</code>
<code>siprouteheaderbase.h</code>	<code>class CSIPRouteHeaderBase</code>
<code>sipsecurityclientheader.h</code>	<code>class CSIPSecurityClientHeader</code>
<code>SipSecurityHeaderBase.h</code>	<code>class CSIPSecurityHeaderBase</code>

SipSecurityServerHeader.h	class CSIPSecurityServerHeader
sipsubscriptionstateheader.h	class CSIPSubscriptionStateHeader
sipsupportedheader.h	class CSIPSupportedHeader
siptoheader.h	class CSIPToHeader
siptokenheaderbase.h	class CSIPTokenHeaderBase
SipUnsignedIntHeaderBase.h	class CSIPUnsignedIntHeaderBase
sipunsupportedheader.h	class CSIPUnsupportedHeader
sipuri.h	class CSIPURI
uribase.h	class CURIBase

3 Index

3.1 List of Constants and Enums

```

/**
 *class CSIPHostPort
 * SIP host types
 */
enum TType
{
    ESIPNoHost=0,
    ESIPIpv4=1,
    ESIPIpv6=2,
    ESIPHostName=3 // fully qualified domain name
};

/**
 * class CSIPResponse
 * Response classes
 */
enum TType
{
    E1XX = 1,
    E2XX,
    E3XX,
    E4XX,
    E5XX,
    E6XX
};

/**
 * * SIP parsing (decoding) errors
 */
enum TErrors
{
    // General errors
    EErrorPreParser=-17750,
    EErrorTokenizer=-17751,
    EErrorInsertAsExtensionHeaderNotAllowed=-17752,
    EErrorInHeaderName=-17753,
    EErrorInHeaderValue=-17754,
    EErrorInRequestLine=-17755,
    EErrorInResponseLine=-17756,
    EErrorInResponseCode=-17757,
    EErrorInSIPVersion=-17758,
    EErrorInSIPMethod=-17759,
    EErrorInReasonPhrase=-17760,
    EErrorInAnyParam=-17761,
    EErrorInAnyParamName=-17762,
    EErrorInAnyParamValue=-17763,
    EErrorDuplicateParam=-17764,
    EErrorInHostPort=-17765,
    EErrorInHost=-17766,
    EErrorInPort=-17767,
    // Via
    EErrorInViaHeader=-17768,
    EErrorInViaSentProtocol=-17769,
    EErrorInViaSentProtocolName=-17770,
    EErrorInViaSentProtocolVersion=-17771,
    EErrorInViaTransport=-17772,
    EErrorInViaParams=-17773,
    EErrorInViaTtl=-17774,
    EErrorInViaMaddr=-17775,
    EErrorInViaReceived=-17776,
    EErrorInViaBranch=-17777,
    EErrorInViaExtensionParam=-17778,
    // From or To

```

```

EErrorInFromOrToAddress=-17779,
EErrorInFromOrToParams=-17780,
EErrorInFromOrToTag=-17781,
EErrorInFromOrToExtensionParam=-17782,
// Contact
EErrorInContactHeader=-17783,
EErrorContactSetToStar=-17784,
EErrorInContactAddress=-17785,
EErrorInContactParams=-17786,
EErrorInContactExpiresParam=-17787,
EErrorInContactQParam=-17788,
EErrorInContactExtensionParam=-17789,
// Call-Id
EErrorInCallIdHeader=-17790,
// CSeq
EErrorInCSeqHeader=-17791,
EErrorInCSeqMethod=-17792,
// Expires
EErrorInExpiresHeader=-17793,
// Max-Forwards
EErrorInMaxForwardsHeader=-17794,
// Content-Type
EErrorInContentTypeHeader=-17795,
EErrorInContentMediaType=-17796,
EErrorInContentMediaSubtype=-17797,
EErrorInContentTypeParams=-17798,
EErrorInContentTypeParam=-17799,
// Content-Length
EErrorInContentLengthHeader=-17800,
// extension-header
EErrorInExtensionHeaderName=-17801,
EErrorInExtensionHeaderValue=-17802,
// sip-address
EErrorInSIPAddress=-17803,
EErrorInDisplayName=-17804,
// URI
EErrorInURI=-17805,
EErrorInURIScheme=-17806,
// SIP(S)-URI
EErrorInSIPURI=-17807,
EErrorInSIPURIUser=-17808,
EErrorInSIPURIPassword=-17809,
EErrorInSIPURIScheme=-17810,
EErrorInSIPURIParams=-17811,
EErrorInSIPURITransport=-17812,
EErrorInSIPURIUserParam=-17813,
EErrorInSIPURIMethod=-17814,
EErrorInSIPURITtl=-17815,
EErrorInSIPURIMaddr=-17816,
EErrorInSIPURILr=-17817,
EErrorInSIPURIExtensionParam=-17818,
EErrorInSIPURIHeaders=-17819,
// Other SIP headers
EErrorInRouteOrRecordRouteHeader=-17820,
EErrorInRouteHeader=-17821,
EErrorInRecordRouteHeader=-17822,
EErrorInMinExpiresHeader=-17823,
EErrorInTokenBaseHeader=-17824,
EErrorInRequireHeader=-17825,
EErrorInContentEncodingHeader=-17826,
EErrorInProxyRequireHeader=-17827,
EErrorInSupportedHeader=-17828,
EErrorInUnsupportedHeader=-17829,
EErrorInAllowHeader=-17830,
EErrorInSecurityHeader=-17831,
EErrorInContentDispositionHeader=-17832,
EErrorInSubscriptionStateHeader=-17833,
EErrorInAcceptHeader=-17834,
EErrorInAcceptEncodingHeader=-17835,

```

```

EErrorInAcceptLanguageHeader= -17836,
EErrorInAcceptParams=-17837,
EErrorInEventHeader=-17838,
EErrorInAllowEventsHeader=-17839,
EErrorInAuthenticationInfoHeader=-17840
};

```

3.2 Alphabetical List of Classes and Functions

3.2.1 CSIPAcceptEncodingHeader

```

IMPORT_C ~CSIPAcceptEncodingHeader ();
IMPORT_C CSIPHeaderBase* CloneL () const;
IMPORT_C const TDesC8& Codings () const;
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8& aValue);
IMPORT_C const TDesC8& FullName () const
IMPORT_C TBool HasQParameter () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream& aReadStream)
IMPORT_C TBool MoreThanOneAllowed () const
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPAcceptEncodingHeader* NewL (const CSIPAcceptEncodingHeader&
aAcceptEncodingHeader)
IMPORT_C static CSIPAcceptEncodingHeader* NewL (const TDesC8& aCodings)
IMPORT_C static CSIPAcceptEncodingHeader* NewLC (const CSIPAcceptEncodingHeader&
aAcceptEncodingHeader)
IMPORT_C static CSIPAcceptEncodingHeader* NewLC (const TDesC8& aCodings)
IMPORT_C TReal QParameter () const
IMPORT_C void SetCodingsL (const TDesC8& aCodings)
IMPORT_C void SetQParameterL (TReal aQValue)

```

3.2.2 CSIPAcceptHeader

```

IMPORT_C ~CSIPAcceptHeader ()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C TInt DeleteMediaParam (const TDesC8& aName)
IMPORT_C const TDesC8& FullName () const
IMPORT_C TBool HasMediaParam (const TDesC8& aName) const
IMPORT_C TBool HasQParameter () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C TBool IsEmpty () const
IMPORT_C const TDesC8& MediaParamValue (const TDesC8& aName) const
IMPORT_C const TDesC8& MediaSubtype () const
IMPORT_C const TDesC8& MediaType () const
IMPORT_C TBool MoreThanOneAllowed () const
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPAcceptHeader* NewL (const CSIPAcceptHeader&
aAcceptHeader)
IMPORT_C static CSIPAcceptHeader* NewL (const TDesC8& aMediaType, const
TDesC8& aMediaSubtype)
IMPORT_C static CSIPAcceptHeader* NewLC (const CSIPAcceptHeader&
aAcceptHeader)
IMPORT_C static CSIPAcceptHeader* NewLC (const TDesC8& aMediaType, const
TDesC8& aMediaSubtype)
IMPORT_C TReal QParameter () const
IMPORT_C void SetMediaParamL (const TDesC8& aName, const TDesC8& aValue)
IMPORT_C void SetMediaSubtypeL (const TDesC8& aMediaSubtype)
IMPORT_C void SetMediaTypeL (const TDesC8& aMediaType)
IMPORT_C void SetQParameterL (TReal aQValue)

```

3.2.3 CSIPAcceptLanguageHeader

```

IMPORT_C ~CSIPAcceptLanguageHeader ()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C TBool HasQParameter () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C const TDesC8& LanguageRange () const
IMPORT_C TBool MoreThanOneAllowed () const
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPAcceptLanguageHeader* NewL (const
CSIPAcceptLanguageHeader& aAcceptLanguageHeader)
IMPORT_C static CSIPAcceptLanguageHeader* NewL (const TDesC8&
aLanguageRange)
IMPORT_C static CSIPAcceptLanguageHeader* NewLC (const
CSIPAcceptLanguageHeader& aAcceptLanguageHeader)
IMPORT_C static CSIPAcceptLanguageHeader* NewLC (const TDesC8&
aLanguageRange)
IMPORT_C TReal QParameter () const
IMPORT_C void SetLanguageRangeL (const TDesC8& aLanguageRange)
IMPORT_C void SetQParameterL (TReal aQValue)

```

3.2.4 CSIPAddress

```

IMPORT_C ~CSIPAddress()
IMPORT_C static CSIPAddress* DecodeL (const TDesC8& aValue)
IMPORT_C const TDesC8& DisplayName () const
IMPORT_C void ExternalizeL (RWriteStream& aWriteStream)
IMPORT_C TBool HasDisplayName () const
IMPORT_C static CSIPAddress* InternalizeL (RReadStream& aReadStream)
IMPORT_C static CSIPAddress* NewL (const CSIPAddress& aSIPAddress)
IMPORT_C static CSIPAddress* NewL (const TDesC8& aDisplayName, CSIPURI*
aURI)
IMPORT_C static CSIPAddress* NewL (CSIPURI* aURI)
IMPORT_C static CSIPAddress* NewLC (const CSIPAddress& aSIPAddress)
IMPORT_C static CSIPAddress* NewLC (const TDesC8& aDisplayName, CSIPURI* aURI)
IMPORT_C static CSIPAddress* NewLC (CSIPURI* aURI)
IMPORT_C TBool operator==(const CSIPAddress& aSIPAddress) const
IMPORT_C void SetDisplayNameL (const TDesC8& aDisplayName)
IMPORT_C void SetURIL (CSIPURI* aURI)
IMPORT_C HBufC8* ToTextLC (TBool aUseAngleBrackets=EFalse) const
IMPORT_C CSIPURI& URI ()
IMPORT_C const CSIPURI& URI () const

```

3.2.5 CSIPAllowEventsHeader

```

IMPORT_C ~CSIPAllowEventsHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C const TDesC8& CompactName () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& EventPackage () const
IMPORT_C const MDesC8Array& EventTemplates () const
IMPORT_C const TDesC8& FullName () const
IMPORT_C TBool HasCompactName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C TBool MoreThanOneAllowed () const
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPAllowEventsHeader* NewL (const
CSIPAllowEventsHeader& aAllowEventsHeader)
IMPORT_C static CSIPAllowEventsHeader* NewL (const TDesC8&
aEventPackage)

```



```

IMPORT_C static CSIPAllowEventsHeader* NewLC (const
CSIPAllowEventsHeader& aAllowEventsHeader)
IMPORT_C static CSIPAllowEventsHeader* NewLC (const TDesC8&
aEventPackage)
IMPORT_C void SetEventPackageL (const TDesC8& aEventPackage)
IMPORT_C void SetEventTemplatesL (const MDesC8Array& aEventTemplates)
IMPORT_C HBufC8* ToTextValueL () const

```

3.2.6 CSIPAllowHeader

```

IMPORT_C ~CSIPAllowHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPAllowHeader* NewL (const CSIPAllowHeader&
aAllowHeader)
IMPORT_C static CSIPAllowHeader* NewL (const TDesC8& aValue)
IMPORT_C static CSIPAllowHeader* NewLC (const CSIPAllowHeader&
aAllowHeader)
IMPORT_C static CSIPAllowHeader* NewLC (const TDesC8& aValue)

```

3.2.7 CSIPContactHeader

```

IMPORT_C ~CSIPContactHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C const TDesC8& CompactName () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C TInt ExpiresParameter () const
IMPORT_C const TDesC8& FullName () const
IMPORT_C TBool HasCompactName () const
IMPORT_C TBool HasExpiresParameter () const
IMPORT_C TBool HasQParameter () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C TBool MoreThanOneAllowed () const
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPContactHeader* NewL ()
IMPORT_C static CSIPContactHeader* NewL (const CSIPContactHeader&
aContactHeader)
IMPORT_C static CSIPContactHeader* NewL (CSIPAddress* aSIPAddress)
IMPORT_C static CSIPContactHeader* NewLC ()
IMPORT_C static CSIPContactHeader* NewLC (const CSIPContactHeader&
aContactHeader)
IMPORT_C static CSIPContactHeader* NewLC (CSIPAddress* aSIPAddress)
IMPORT_C TBool operator==(const CSIPContactHeader& aContactHeader) const
IMPORT_C TReal QParameter () const
IMPORT_C void SetExpiresParameterL (TInt aExpiresParam)
IMPORT_C void SetQParameterL (TReal aQValue)
IMPORT_C void SetSIPAddressL (CSIPAddress* aSIPAddress)
IMPORT_C CSIPAddress* SIPAddress ()
IMPORT_C const CSIPAddress* SIPAddress () const
IMPORT_C TBool Star () const

```

3.2.8 CSIPContentDispositionHeader

```

IMPORT_C ~CSIPContentDispositionHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& DispType () const
IMPORT_C const TDesC8& FullName () const

```

```

IMPORT_C const TDesC8& HandlingParam () const
IMPORT_C TBool HasHandlingParam () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPContentDispositionHeader* NewL (const
CSIPContentDispositionHeader& aContentDispositionHeader)
IMPORT_C static CSIPContentDispositionHeader* NewL (const TDesC8&
aDispType)
IMPORT_C static CSIPContentDispositionHeader* NewL (const TDesC8&
aDispType, const TDesC8& aHandlingParam)
IMPORT_C static CSIPContentDispositionHeader* NewLC (const
CSIPContentDispositionHeader& aContentDispositionHeader)
IMPORT_C static CSIPContentDispositionHeader* NewLC (const TDesC8&
aDispType)
IMPORT_C static CSIPContentDispositionHeader* NewLC (const TDesC8&
aDispType, const TDesC8& aHandlingParam)
IMPORT_C void SetDispTypeL (const TDesC8& aDispType)
IMPORT_C void SetHandlingParamL (const TDesC8& aHandlingParam)

```

3.2.9 CSIPContentEncodingHeader

```

IMPORT_C ~CSIPContentEncodingHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C const TDesC8& CompactName () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C TBool HasCompactName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPContentEncodingHeader* NewL (const
CSIPContentEncodingHeader& aContentEncodingHeader)
IMPORT_C static CSIPContentEncodingHeader* NewL (const TDesC8& aValue)
IMPORT_C static CSIPContentEncodingHeader* NewLC (const
CSIPContentEncodingHeader& aContentEncodingHeader)
IMPORT_C static CSIPContentEncodingHeader* NewLC (const TDesC8& aValue)

```

3.2.10 CSIPContentTypeHeader

```

IMPORT_C ~CSIPContentTypeHeader ()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C const TDesC8& CompactName () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C TBool HasCompactName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C const TDesC8& MediaSubtype () const
IMPORT_C const TDesC8& MediaType () const
IMPORT_C TBool MoreThanOneAllowed () const
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPContentTypeHeader* NewL (const
CSIPContentTypeHeader& aContentTypeHeader)
IMPORT_C static CSIPContentTypeHeader* NewL (const TDesC8& aMediaType,
const TDesC8& aMediaSubtype)
IMPORT_C static CSIPContentTypeHeader* NewLC (const
CSIPContentTypeHeader& aContentTypeHeader)
IMPORT_C static CSIPContentTypeHeader* NewLC (const TDesC8& aMediaType
const TDesC8& aMediaSubtype)
IMPORT_C void SetMediaSubtypeL (const TDesC8& aMediaSubtype)
IMPORT_C void SetMediaTypeL (const TDesC8& aMediaType)

```

3.2.11 CSIPSeqHeader

```

IMPORT_C ~CSIPSeqHeader ();
IMPORT_C CSIPHeaderBase* CloneL () const;
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue);
IMPORT_C const TDesC8& FullName () const;
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream);
IMPORT_C const TDesC8& Method () const;
IMPORT_C static const TDesC8& Name ();
IMPORT_C static CSIPSeqHeader* NewL (const CSIPSeqHeader&
aCSeqHeader);
IMPORT_C static CSIPSeqHeader* NewL (TUint aSeq, const TDesC8&
aMethod);
IMPORT_C static CSIPSeqHeader* NewLC (const CSIPSeqHeader&
aCSeqHeader);
IMPORT_C static CSIPSeqHeader* NewLC (TUint aSeq, const TDesC8&
aMethod);
IMPORT_C TBool operator==(const CSIPSeqHeader& aCSeqHeader);
IMPORT_C TUint Seq () const;
IMPORT_C void SetMethodL (const TDesC8& aMethod);
IMPORT_C void SetSeq (TUint aSeq);
IMPORT_C HBufC8* ToTextValueL () const;

```

3.2.12 CSIPEventHeader

```

IMPORT_C ~CSIPEventHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C const TDesC8& CompactName () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& EventPackage () const
IMPORT_C const MDesC8Array&EventTemplates () const
IMPORT_C const TDesC8& FullName () const
IMPORT_C TBool HasCompactName () const
IMPORT_C TBool HasIdParam () const
IMPORT_C const TDesC8& IdParam () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPEventHeader* NewL (const CSIPEventHeader&
aEventHeader)
IMPORT_C static CSIPEventHeader* NewL (const TDesC8& aEventPackage)
IMPORT_C static CSIPEventHeader* NewLC (const CSIPEventHeader&
aEventHeader)
IMPORT_C static CSIPEventHeader* NewLC (const TDesC8& aEventPackage)
IMPORT_C TBool operator==(const CSIPEventHeader& aEventHeader) const
IMPORT_C void SetEventPackageL (const TDesC8& aEventPackage)
IMPORT_C void SetEventTemplatesL (const MDesC8Array& aEventTemplates)
IMPORT_C void SetIdParamL (const TDesC8& aIdParam)

```

3.2.13 CSIPExpiresHeader

```

IMPORT_C ~CSIPExpiresHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C CSIPExpiresHeader (TUint aValue)
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()

```

3.2.14 CSIPExtensionHeader

```

IMPORT_C ~CSIPExtensionHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C const TDesc8& CompactName () const
IMPORT_C TBool ExternalizeSupported () const
IMPORT_C const TDesc8& FullName () const
IMPORT_C TBool HasCompactName () const
IMPORT_C static CSIPExtensionHeader* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C TBool IsExtensionHeader () const
IMPORT_C TBool MoreThanOneAllowed () const
IMPORT_C static CSIPExtensionHeader* NewL (const CSIPExtensionHeader&
aExtensionHeader)
IMPORT_C static CSIPExtensionHeader* NewL (const TDesc8& aName, const
TDesc8& aValue)
IMPORT_C static CSIPExtensionHeader* NewLC (const CSIPExtensionHeader&
aExtensionHeader)
IMPORT_C static CSIPExtensionHeader* NewLC (const TDesc8& aName, const
TDesc8& aValue)
IMPORT_C TBool operator==(const CSIPExtensionHeader& aExtensionHeader)
IMPORT_C void SetNameL (const TDesc8& aName)
IMPORT_C void SetValueL (const TDesc8& aValue)
IMPORT_C HBufC8* ToTextValueL () const
IMPORT_C const TDesc8& Value () const

```

3.2.15 CSIPFromHeader

```

IMPORT_C ~CSIPFromHeader ()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C const TDesc8& CompactName () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesc8&
aValue)
IMPORT_C const TDesc8& FullName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesc8& Name ()
IMPORT_C static CSIPFromHeader* NewL (const CSIPFromToHeaderBase&
aFromToHeaderBase)
IMPORT_C static CSIPFromHeader* NewL (CSIPAddress* aSIPAddress)
IMPORT_C static CSIPFromHeader* NewL (CSIPAddress* aSIPAddress, const
TDesc8& aTag)
IMPORT_C static CSIPFromHeader* NewLC (const CSIPFromToHeaderBase&
aFromToHeaderBase)
IMPORT_C static CSIPFromHeader* NewLC (CSIPAddress* aSIPAddress)
IMPORT_C static CSIPFromHeader* NewLC (CSIPAddress* aSIPAddress, const
TDesc8& aTag)

```

3.2.16 CSIPFromToHeaderBase

```

IMPORT_C virtual ~CSIPFromToHeaderBase()
IMPORT_C TBool HasCompactName () const
IMPORT_C TBool HasTag () const
IMPORT_C TBool operator==(const CSIPFromToHeaderBase&
aSIPFromToHeaderBase) const
IMPORT_C void SetSIPAddressL (CSIPAddress* aSIPAddress)
IMPORT_C void SetTagL (const TDesc8& aTag)
IMPORT_C CSIPAddress& SIPAddress()
IMPORT_C const CSIPAddress& SIPAddress() const
IMPORT_C const TDesc8& Tag () const

```

3.2.17 CSIPHeaderBase

```

IMPORT_C virtual ~CSIPHeaderBase();
IMPORT_C virtual CSIPHeaderBase* CloneL () const = 0
IMPORT_C virtual const TDesc8& CompactName () const

```

```

IMPORT_C void ExternalizeL (RWriteStream& aWriteStream, TBool
aAddName=ETrue) const
IMPORT_C virtual TBool ExternalizeSupported () const
IMPORT_C virtual const TDesC8& FullName () const = 0
IMPORT_C virtual TBool HasCompactName () const
IMPORT_C virtual TBool IsExtensionHeader () const
IMPORT_C virtual TBool MoreThanOneAllowed() const
IMPORT_C static void PushLC (RPointerArray<CSIPHeaderBase>* aArray)
IMPORT_C HBufC8* ToTextL () const
IMPORT_C HBufC8* ToTextLC () const
IMPORT_C virtual HBufC8* ToTextValueL () const = 0
IMPORT_C HBufC8* ToTextValueLC () const

```

3.2.18 CSIPHostPort

```

IMPORT_C ~CSIPHostPort()
IMPORT_C static CSIPHostPort* DecodeL (const TDesC8& aValue)
IMPORT_C TInt DeletePort ()
IMPORT_C TBool HasHost () const
IMPORT_C TBool HasPort () const
IMPORT_C const TDesC8& Host () const
IMPORT_C TType HostType () const
IMPORT_C static CSIPHostPort* NewL ()
IMPORT_C static CSIPHostPort* NewL (const CSIPHostPort& aHostPort)
IMPORT_C static CSIPHostPort* NewL (const TDesC8& aHost)
IMPORT_C static CSIPHostPort* NewL (const TDesC8& aHost, TInt aPort)
IMPORT_C static CSIPHostPort* NewLC ()
IMPORT_C static CSIPHostPort* NewLC (const CSIPHostPort& aHostPort)
IMPORT_C static CSIPHostPort* NewLC (const TDesC8& aHost)
IMPORT_C static CSIPHostPort* NewLC (const TDesC8& aHost, TInt aPort)
IMPORT_C TBool operator==(const CSIPHostPort& aHostPort) const
IMPORT_C TInt Port () const
IMPORT_C void SetHostL (const TDesC8& aHost)
IMPORT_C void SetPort (TInt aPort)
IMPORT_C HBufC8* ToTextLC () const

```

3.2.19 CSIPParameterHeaderBase

```

IMPORT_C virtual ~CSIPParameterHeaderBase()
IMPORT_C TInt DeleteExtensionParam (const TDesC8& aName)
IMPORT_C const TDesC8& ExtensionParamValue (const TDesC8& aName) const
IMPORT_C TBool HasExtensionParam (const TDesC8& aName) const
IMPORT_C void SetExtensionParamL (const TDesC8& aName)
IMPORT_C void SetExtensionParamL (const TDesC8& aName, const TDesC8&
aValue)
IMPORT_C HBufC8* ToTextValueL () const

```

3.2.20 CSIPProxyRequireHeader

```

IMPORT_C ~CSIPProxyRequireHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPProxyRequireHeader* NewL (const
CSIPProxyRequireHeader& aProxyRequireHeader)
IMPORT_C static CSIPProxyRequireHeader* NewL (const TDesC8& aValue)
IMPORT_C static CSIPProxyRequireHeader* NewLC (const
CSIPProxyRequireHeader& aProxyRequireHeader)
IMPORT_C static CSIPProxyRequireHeader* NewLC (const TDesC8& aValue)
IMPORT_C TBool operator==(const CSIPProxyRequireHeader&
aProxyRequireHeader) const

```

3.2.21 CSIPRequireHeader

```

IMPORT_C ~CSIPRequireHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPRequireHeader* NewL (const CSIPRequireHeader&
aRequireHeader)
IMPORT_C static CSIPRequireHeader* NewL (const TDesC8& aValue)
IMPORT_C static CSIPRequireHeader* NewLC (const CSIPRequireHeader&
aRequireHeader)
IMPORT_C static CSIPRequireHeader* NewLC (const TDesC8& aValue)
IMPORT_C TBool operator==(const CSIPRequireHeader& aRequireHeader) const

```

3.2.22 CSIPRouteHeader

```

IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C void ExternalizeValueL (RWriteStream& aWriteStream) const
IMPORT_C const TDesC8& FullName () const
IMPORT_C ~CSIPRouteHeader()
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPRouteHeader* NewL (const CSIPRouteHeaderBase&
aRouteHeaderBase)
IMPORT_C static CSIPRouteHeader* NewL (CSIPAddress* aSIPAddress)
IMPORT_C static CSIPRouteHeader* NewLC (const CSIPRouteHeaderBase&
aRouteHeaderBase)
IMPORT_C static CSIPRouteHeader* NewLC (CSIPAddress* aSIPAddress)

```

3.2.23 CSIPRouteHeaderBase

```

IMPORT_C virtual ~CSIPRouteHeaderBase()
IMPORT_C TBool MoreThanOneAllowed () const
IMPORT_C TBool operator==(const CSIPRouteHeaderBase& aRouteHeaderBase)
const
IMPORT_C void SetSIPAddressL (CSIPAddress* aSIPAddress)
IMPORT_C CSIPAddress& SIPAddress ()
IMPORT_C const CSIPAddress& SIPAddress () const

```

3.2.24 CSIPSecurityClientHeader

```

IMPORT_C ~CSIPSecurityClientHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPSecurityClientHeader* NewL (const
CSIPSecurityHeaderBase& aSecurityHeader)
IMPORT_C static CSIPSecurityClientHeader* NewL (const TDesC8&
aMechanismName)
IMPORT_C static CSIPSecurityClientHeader* NewLC (const
CSIPSecurityHeaderBase& aSecurityHeader)
IMPORT_C static CSIPSecurityClientHeader* NewLC (const TDesC8&
aMechanismName)

```


3.2.25 CSIPSecurityHeaderBase

```

IMPORT_C ~CSIPSecurityHeaderBase()
IMPORT_C const TDesC8& DigestAlgorithmParam () const
IMPORT_C const TDesC8& DigestQopParam () const
IMPORT_C const TDesC8& DigestVerifyParam () const
IMPORT_C TBool HasCompactName () const
IMPORT_C TBool HasDigestAlgorithmParam () const
IMPORT_C TBool HasDigestQopParam () const
IMPORT_C TBool HasDigestVerifyParam () const
IMPORT_C TBool HasPreferenceParam () const
IMPORT_C const TDesC8& MechanismName () const
IMPORT_C TBool MoreThanOneAllowed () const
IMPORT_C TReal PreferenceParam () const
IMPORT_C void SetDigestAlgorithmParamL (const TDesC8&
aDigestAlgorithmValue)
IMPORT_C void SetDigestQopParamL (const TDesC8& aDigestQopValue)
IMPORT_C void SetDigestVerifyParamL (const TDesC8& aDigestVerifyValue)
IMPORT_C void SetMechanismNameL (const TDesC8& aMechanismName)
IMPORT_C void SetPreferenceParamL (TReal aPreferenceValue)

```

3.2.26 CSIPSecurityServerHeader

```

IMPORT_C ~CSIPSecurityServerHeader()
IMPORT_C CSIPHeaderBase* CloneL () const ;
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPSecurityServerHeader* NewL (const
CSIPSecurityHeaderBase& aSecurityHeader)
IMPORT_C static CSIPSecurityServerHeader* NewL (const TDesC8&
aMechanismName)
IMPORT_C static CSIPSecurityServerHeader* NewLC (const
CSIPSecurityHeaderBase& aSecurityHeader)
IMPORT_C static CSIPSecurityServerHeader* NewLC (const TDesC8&
aMechanismName)

```

3.2.27 CSIPSubscriptionStateHeader

```

IMPORT_C ~CSIPSubscriptionStateHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C TUint ExpiresParameter () const
IMPORT_C const TDesC8& FullName () const
IMPORT_C TBool HasExpiresParameter () const
IMPORT_C TBool HasReasonParam () const
IMPORT_C TBool HasRetryAfterParameter () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPSubscriptionStateHeader* NewL (const
CSIPSubscriptionStateHeader& aSubscriptionStateHeader)
IMPORT_C static CSIPSubscriptionStateHeader* NewL (const TDesC8&
aSubStateValue)
IMPORT_C static CSIPSubscriptionStateHeader* NewLC (const
CSIPSubscriptionStateHeader& aSubscriptionStateHeader)
IMPORT_C static CSIPSubscriptionStateHeader* NewLC (const TDesC8&
aSubStateValue)
IMPORT_C const TDesC8& ReasonParam () const
IMPORT_C TUint RetryAfterParameter () const
IMPORT_C void SetExpiresParameterL (TUint aExpiresParam)
IMPORT_C void SetReasonParamL (const TDesC8& aReasonParam)
IMPORT_C void SetRetryAfterParameterL (TUint aRetryAfterParam)

```

```
IMPORT_C void SetSubStateValueL (const TDesC8& aSubStateValue)
IMPORT_C const TDesC8& SubStateValue () const
```

3.2.28 CSIPSupportedHeader

```
IMPORT_C ~CSIPSupportedHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C const TDesC8& CompactName () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C TBool HasCompactName () const
IMPORT_C TBool HasValue () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPSupportedHeader* NewL (const CSIPSupportedHeader&
aSupportedHeader)
IMPORT_C static CSIPSupportedHeader* NewL (const TDesC8& aValue)
IMPORT_C static CSIPSupportedHeader* NewLC (const CSIPSupportedHeader&
aSupportedHeader)
IMPORT_C static CSIPSupportedHeader* NewLC (const TDesC8& aValue)
IMPORT_C TBool operator==(const CSIPSupportedHeader& aSupportedHeader)
const
```

3.2.29 CSIPToHeader

```
IMPORT_C ~CSIPToHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C const TDesC8& CompactName () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPToHeader* NewL (const CSIPFromToHeaderBase&
aFromToHeaderBase)
IMPORT_C static CSIPToHeader* NewL (CSIPAddress* aSIPAddress)
IMPORT_C static CSIPToHeader* NewL (CSIPAddress* aSIPAddress, const
TDesC8& aTag)
IMPORT_C static CSIPToHeader* NewLC (const CSIPFromToHeaderBase&
aFromToHeaderBase)
IMPORT_C static CSIPToHeader* NewLC (CSIPAddress* aSIPAddress)
IMPORT_C static CSIPToHeader* NewLC (CSIPAddress* aSIPAddress, const
TDesC8& aTag)
```

3.2.30 CSIPTokenHeaderBase

```
IMPORT_C ~CSIPTokenHeaderBase()
IMPORT_C TBool MoreThanOneAllowed () const
IMPORT_C void SetValueL (const TDesC8& aValue)
IMPORT_C HBufC8* ToTextValueL () const
IMPORT_C const TDesC8& Value () const
```

3.2.31 CSIPUnsignedIntHeaderBase

```
IMPORT_C ~CSIPUnsignedIntHeaderBase()
IMPORT_C void SetValue (TUint aValue)
IMPORT_C HBufC8* ToTextValueL () const
IMPORT_C TUint Value () const
```


3.2.32 CSIPUnsupportedHeader

```

IMPORT_C ~CSIPUnsupportedHeader()
IMPORT_C CSIPHeaderBase* CloneL () const
IMPORT_C static RPointerArray<CSIPHeaderBase> DecodeL (const TDesC8&
aValue)
IMPORT_C const TDesC8& FullName () const
IMPORT_C static CSIPHeaderBase* InternalizeValueL (RReadStream&
aReadStream)
IMPORT_C static const TDesC8& Name ()
IMPORT_C static CSIPUnsupportedHeader* NewL (const
CSIPUnsupportedHeader& aUnsupportedHeader)
IMPORT_C static CSIPUnsupportedHeader* NewL (const TDesC8& aValue)
IMPORT_C static CSIPUnsupportedHeader* NewLC (const
CSIPUnsupportedHeader& aUnsupportedHeader)
IMPORT_C static CSIPUnsupportedHeader* NewLC (const TDesC8& aValue)
IMPORT_C TBool operator==(const CSIPUnsupportedHeader&
aUnsupportedHeader) const

```

3.2.33 CSIPURI

```

IMPORT_C ~CSIPURI()
IMPORT_C void AddURIHeaderL (const TDesC8& aName, const TDesC8& aValue)
IMPORT_C CURIBase* CloneL () const
IMPORT_C static CSIPURI* DecodeL (const TDesC8& aValue)
IMPORT_C TInt DeleteLrParam ()
IMPORT_C TInt DeleteMaddrParam ()
IMPORT_C TInt DeleteMethodParam ()
IMPORT_C TInt DeleteOtherURIParam (const TDesC8& aName)
IMPORT_C void DeletePassword ()
IMPORT_C TInt DeleteTransportParam ()
IMPORT_C TInt DeleteTtlParam ()
IMPORT_C void DeleteUserInfo ()
IMPORT_C TInt DeleteUserParam ()
IMPORT_C void ExternalizeL (RWriteStream& aWriteStream) const
IMPORT_C TBool HasLrParam () const
IMPORT_C TBool HasMaddrParam () const
IMPORT_C TBool HasMethodParam () const
IMPORT_C TBool HasOtherURIParam (const TDesC8& aName) const
IMPORT_C TBool HasParams () const
IMPORT_C TBool HasPassword () const
IMPORT_C TBool HasTransportParam () const
IMPORT_C TBool HasTtlParam () const
IMPORT_C TBool HasURIHeaders (const TDesC8& aName) const
IMPORT_C TBool HasUser () const
IMPORT_C TBool HasUserParam () const
IMPORT_C CSIPHostPort& HostPort ()
IMPORT_C const CSIPHostPort& HostPort () const
IMPORT_C static CSIPURI* InternalizeL (RReadStream& aReadStream)
IMPORT_C TBool IsSIPSURI () const
IMPORT_C TBool IsSIPURI () const
IMPORT_C const TDesC8& MaddrParam () const
IMPORT_C const TDesC8& MethodParam () const
IMPORT_C static CSIPURI* NewL (const CSIPURI& aSIPURI)
IMPORT_C static CSIPURI* NewL (const TDesC8& aUser, CSIPHostPort*
aHostPort, TBool aIsSIPSUri=EFalse)
IMPORT_C static CSIPURI* NewL (CSIPHostPort* aHostPort, TBool
aIsSIPSUri=EFalse)
IMPORT_C static CSIPURI* NewLC (const CSIPURI& aSIPURI)
IMPORT_C static CSIPURI* NewLC (const TDesC8& aUser, CSIPHostPort*
aHostPort, TBool aIsSIPSUri=EFalse)
IMPORT_C static CSIPURI* NewLC (CSIPHostPort* aHostPort, TBool
aIsSIPSUri=EFalse)
IMPORT_C TBool operator==(const CURIBase& aURI) const
IMPORT_C const TDesC8& OtherURIParamValue (const TDesC8& aName) const
IMPORT_C const TDesC8& Password () const
IMPORT_C void SetHostPortL (CSIPHostPort* aHostPort)
IMPORT_C void SetLrParamL ()

```

```

IMPORT_C void SetMaddrParamL (const TDesC8& aValue)
IMPORT_C void SetMethodParamL (const TDesC8& aValue)
IMPORT_C void SetOtherURIPParamL (const TDesC8& aName)
IMPORT_C void SetOtherURIPParamL (const TDesC8& aName, const TDesC8&
aValue)
IMPORT_C void SetPasswordL (const TDesC8& aPassword)
IMPORT_C void SetTransportParamL (const TDesC8& aValue)
IMPORT_C void SetTtlParamL (TUint aValue)
IMPORT_C void SetUserL (const TDesC8& aUser)
IMPORT_C void SetUserParamL (const TDesC8& aValue)
IMPORT_C HBufC8*ToTextL () const
IMPORT_C const TDesC8& TransportParam () const
IMPORT_C TUint TtlParam () const
IMPORT_C CPtrC8Array* URIHeaderValuesL (const TDesC8& aName) const
IMPORT_C const TDesC8&User () const
IMPORT_C const TDesC8& UserParam () const

```

3.2.34 CURIBase

```

IMPORT_C virtual ~CURIBase ()
IMPORT_C virtual CURIBase*CloneL () const = 0
IMPORT_C virtual TBool IsSIPURI () const = 0
IMPORT_C virtual TBool operator==(const CURIBase& aURI) const = 0
IMPORT_C const TDesC8& Scheme () const
IMPORT_C virtual HBufC8* ToTextL () const = 0

```

4 Terms and Abbreviations

Term or abbreviation	Meaning
API	Application Programming Interface
BNF	Backus-Naur Form. Formal notation used for expressing the concrete textual syntax of a language Backus-Naur form
DLL	Dynamic Link Library
SIP	Session Initiation Protocol

5 References

- [1] SIP: Session Initiation Protocol, RFC 3261, <http://www.ietf.org/rfc/rfc3261.txt?number=3261>